Linux Systems and Performance

PART 1: Linux Execution Model

Week 2: Exceptional Control Flow: Interrupts and Signals

Sungyong Park, Ph.D.

Data-Intensive AI Computing and Systems (DISCOS) Laboratory

Department of Computer Science and Engineering, Sogang University

URL: https://discos.sogang.ac.kr

E-mail: parksy@sogang.ac.kr

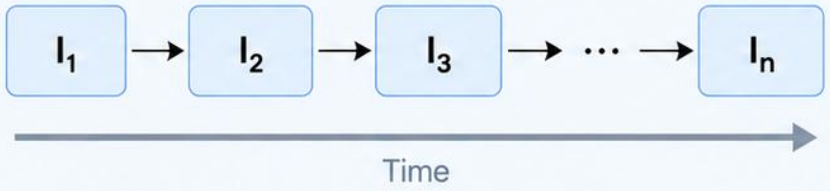


From Normal Control Flow to Exceptional Control Flow (ECF)

WEEK 1: NORMAL CONTROL FLOW

We assumed...

"The CPU executes the next instruction as directed by the program."

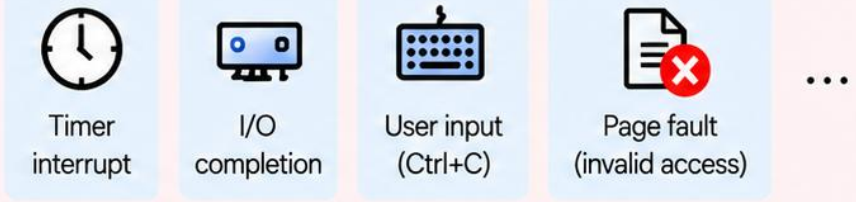


Program-directed control flow
Sequential execution, branches, function calls

But real systems must react to events that change the normal flow.

WEEK 2: EXCEPTIONAL CONTROL FLOW (ECF)

Real systems must respond to events such as:



These events can trigger exceptional control flow, transferring control through the OS/kernel.

Example we already know from Week 1:

Function call & return



The program explicitly transfers control according to its code.

In this week, we study **Exceptional Control Flow (ECF):**
how the system reacts to events while preserving stability, responsiveness, and correct behavior.

Why Do We Need to Learn Exceptional Control Flow (ECF)?

In a running webOS system, many events occur asynchronously while applications are executing.
The system must **immediately handle** these events and then **return** to the interrupted activity or take another action.

1) Remote Button Press



User presses remote button
↓
Hardware generates **interrupt (IRQ)**
↓
Application receives event

An **input event** is delivered to the application.

2) Network Packet Arrives



Packet arrives at NIC
↓
Hardware generates **interrupt (IRQ)**
↓
Socket becomes ready (application notified)

A **network event** is delivered to the application.

3) Invalid Memory Access



CPU detects a **fault** (e.g., page fault)
↓
Kernel handles the fault
↓
Deliver **signal** (e.g., SIGSEGV)

A fault is reported to the process via a **signal**.

4) Timer / Scheduling Event



Timer interrupt occurs
↓
Scheduler runs
↓
Context switch / Process accounting

Enables **time-based activities** and fair CPU sharing.

These are common events in webOS:



User input



Network communication



Application faults



System timing & scheduling

...

Exceptional Control Flow is the mechanism that allows the system to react to these events while preserving stability, responsiveness, and correct behavior.

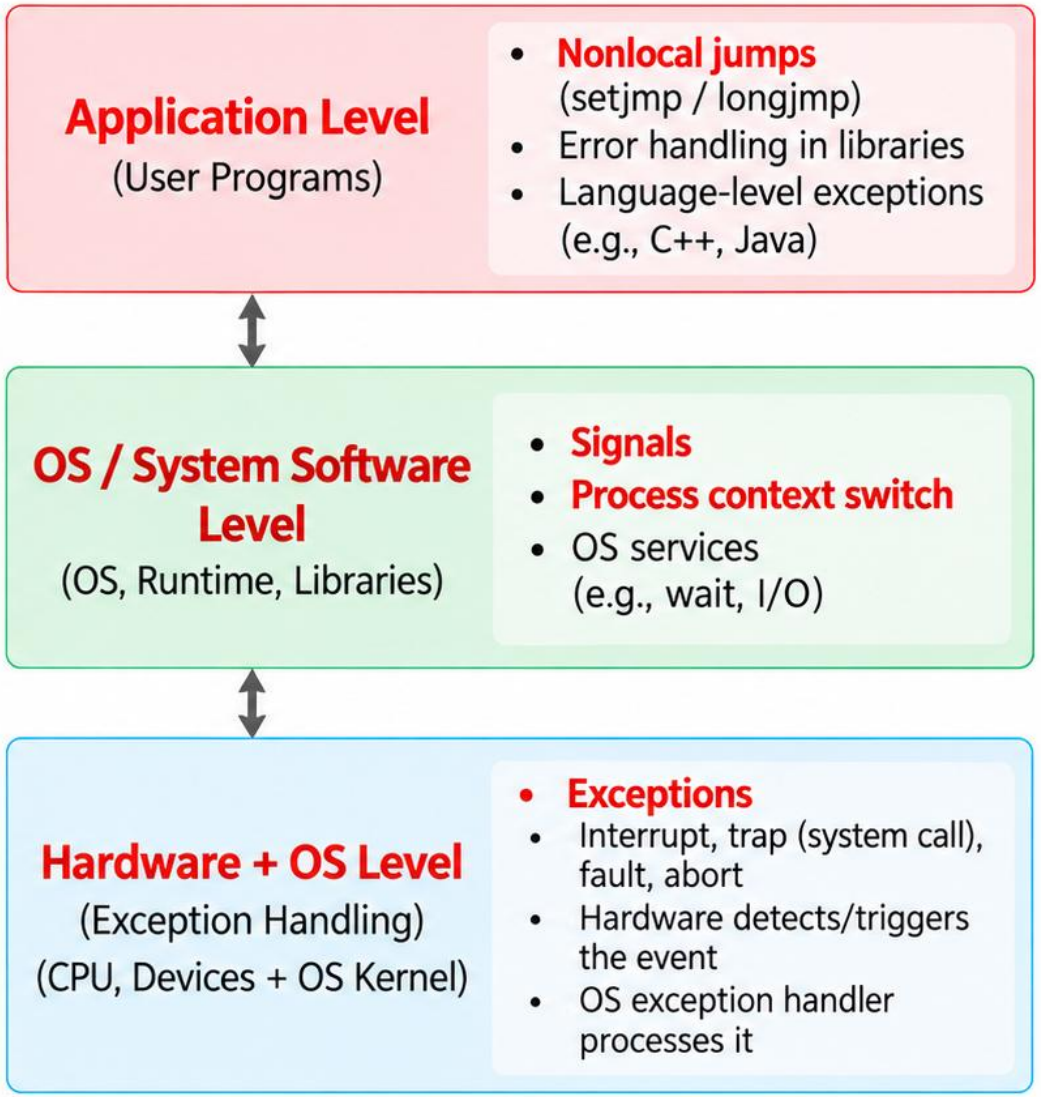
PART 1: Linux Execution Model

- **Week 2**

- **Exceptional Control Flow (ECF)**
  - How can normal control flow change?
- **Interrupts and Device Interface**
  - How does the CPU interact with hardware events?
- **Signals**
  - How does Linux deliver events to processes?

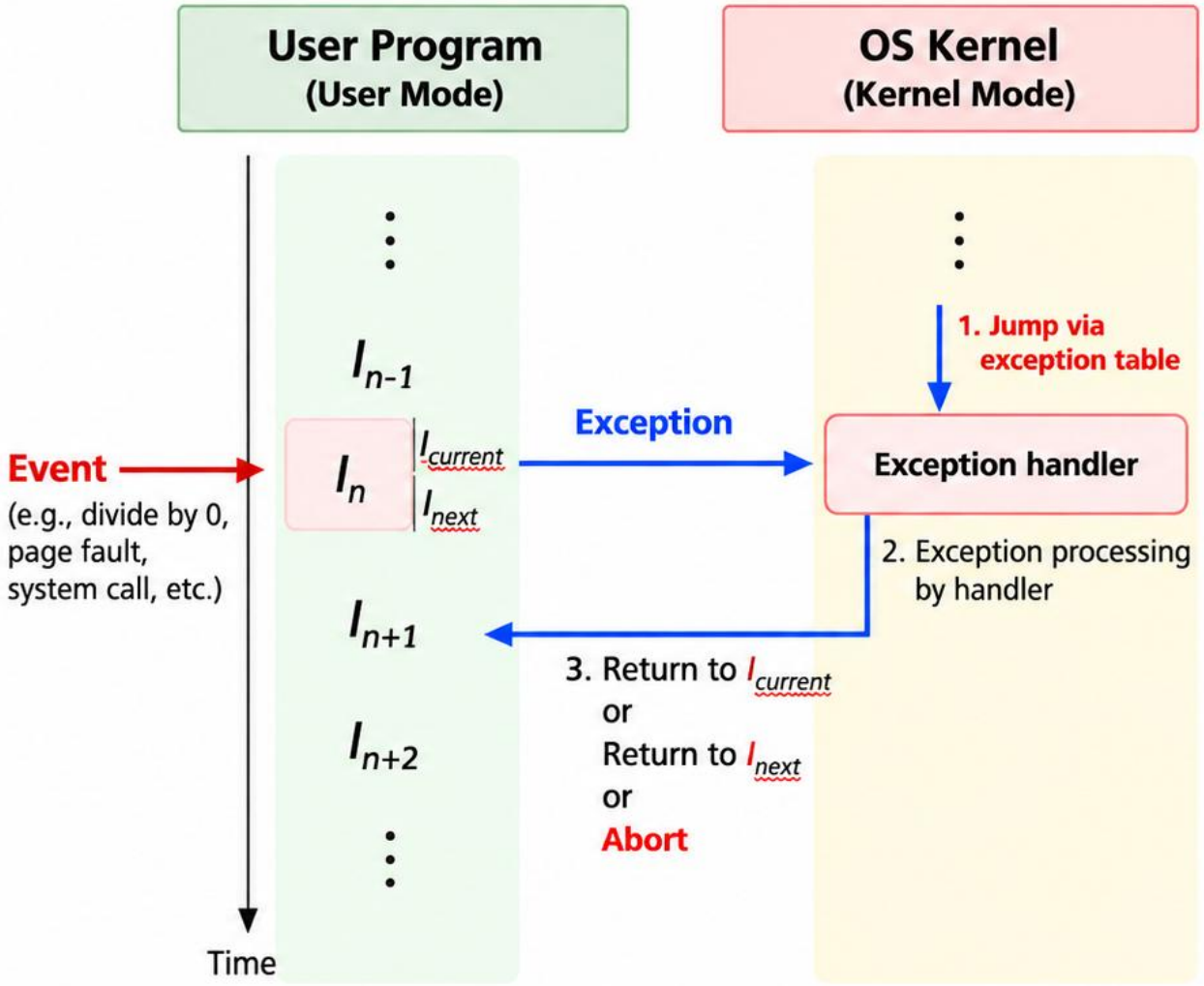
When Does Exceptional Control Flow (ECF) Happen?

- **ECF exists at all levels of a computer system**
- **Low-level mechanisms**
  - **Exceptions**
    - An exception is a change in control flow in response to ***system events***
    - Implemented using a combination of hardware and OS software
    - Examples: interrupt, trap (system call), fault, abort
- **High-level mechanisms**
  - **Process context switch**
    - Implemented by OS software (e.g., scheduler)
    - Often triggered by a hardware timer (e.g., timer interrupt)
  - **Signals**
    - Implemented by OS software
  - **Nonlocal jumps**: `setjmp()` and `longjmp()`
    - Implemented by C runtime library



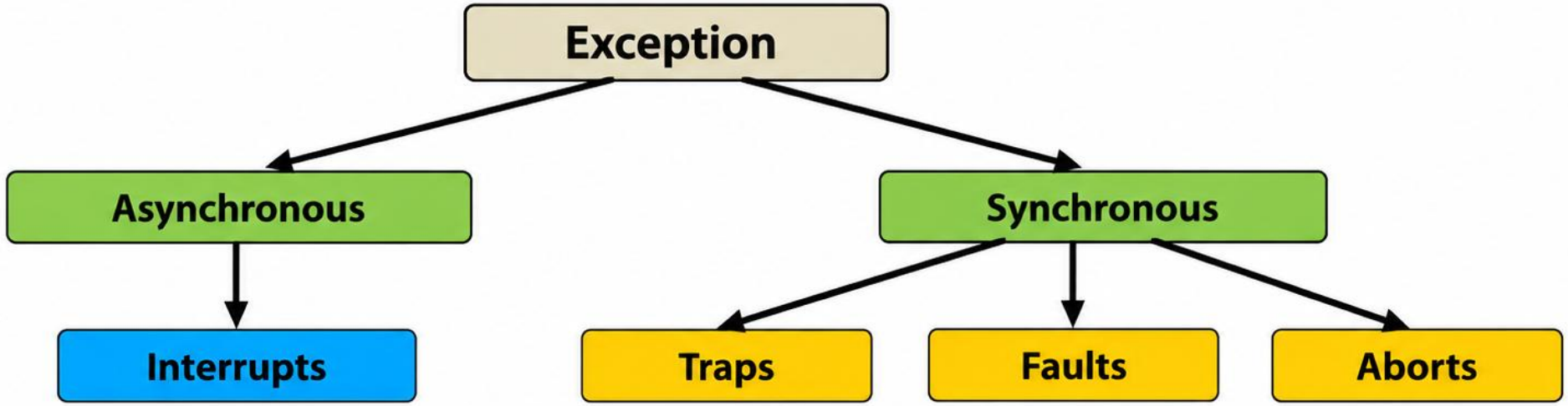
Exceptions

- An ***exception*** is a transfer of control to the **OS kernel** in response to **system events**
- **Examples of events:** divide by 0, page fault, system call, device interrupt, etc.
- Kernel is the memory-resident part of the OS
- Jump to the exception handler via **exception table**
- Handler either returns to the "***current***" instruction, returns to the "***next***" instruction, or "***abort***"



AI-generated illustration using ChatGPT (OpenAI)

Classes of Exceptions



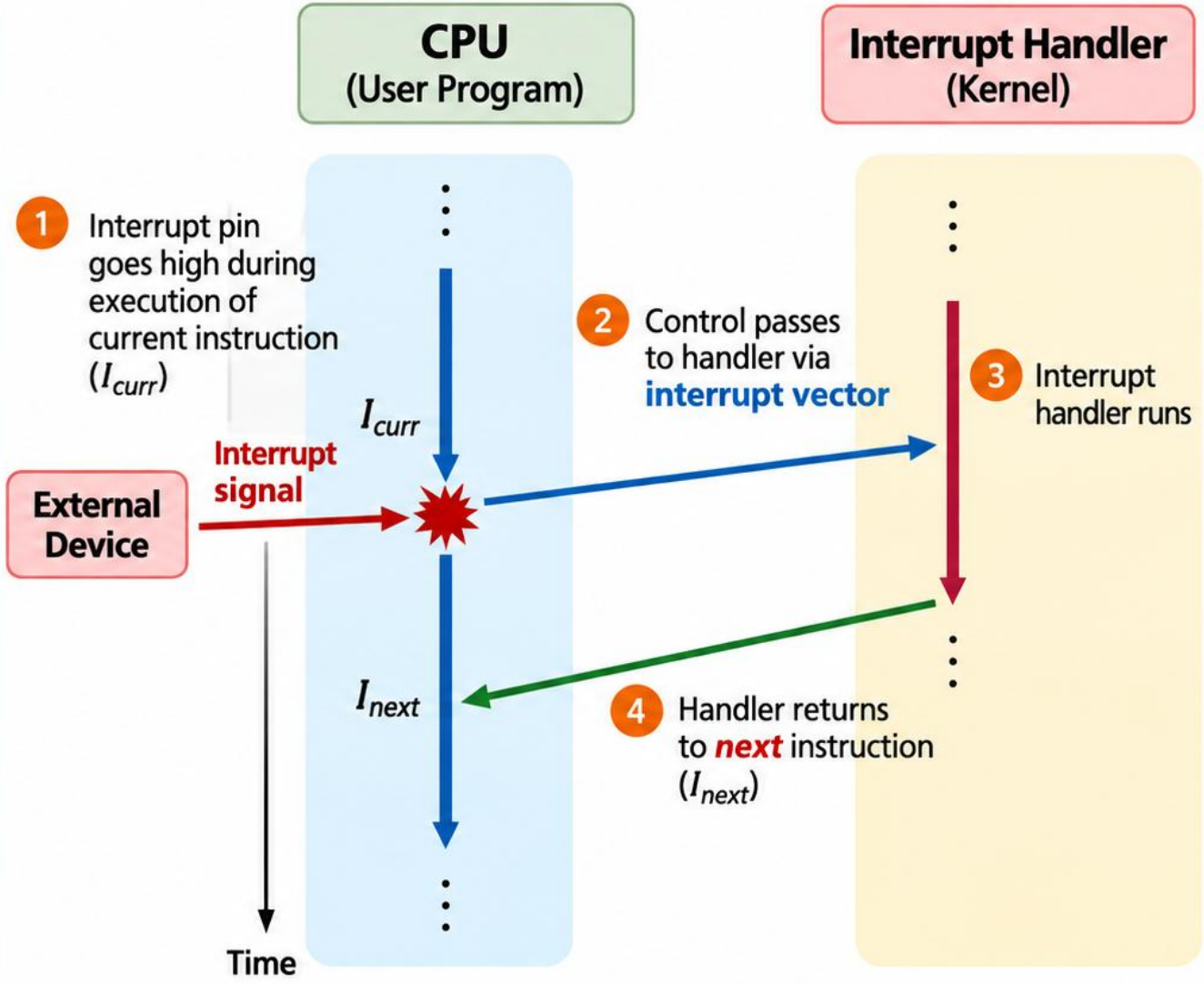
Class	Cause	Async/sync	Return behavior
Interrupt	Signal from I/O device	Async	Always returns to next instruction
Trap	Intentional exception	Sync	Always returns to next instruction
Fault	Potentially recoverable error	Sync	Might return to current instruction
Abort	Nonrecoverable error	Sync	Never returns

Asynchronous exceptions occur as a result of events in I/O devices that are external to the processor.

Synchronous exceptions occur as a direct result of executing an instruction.

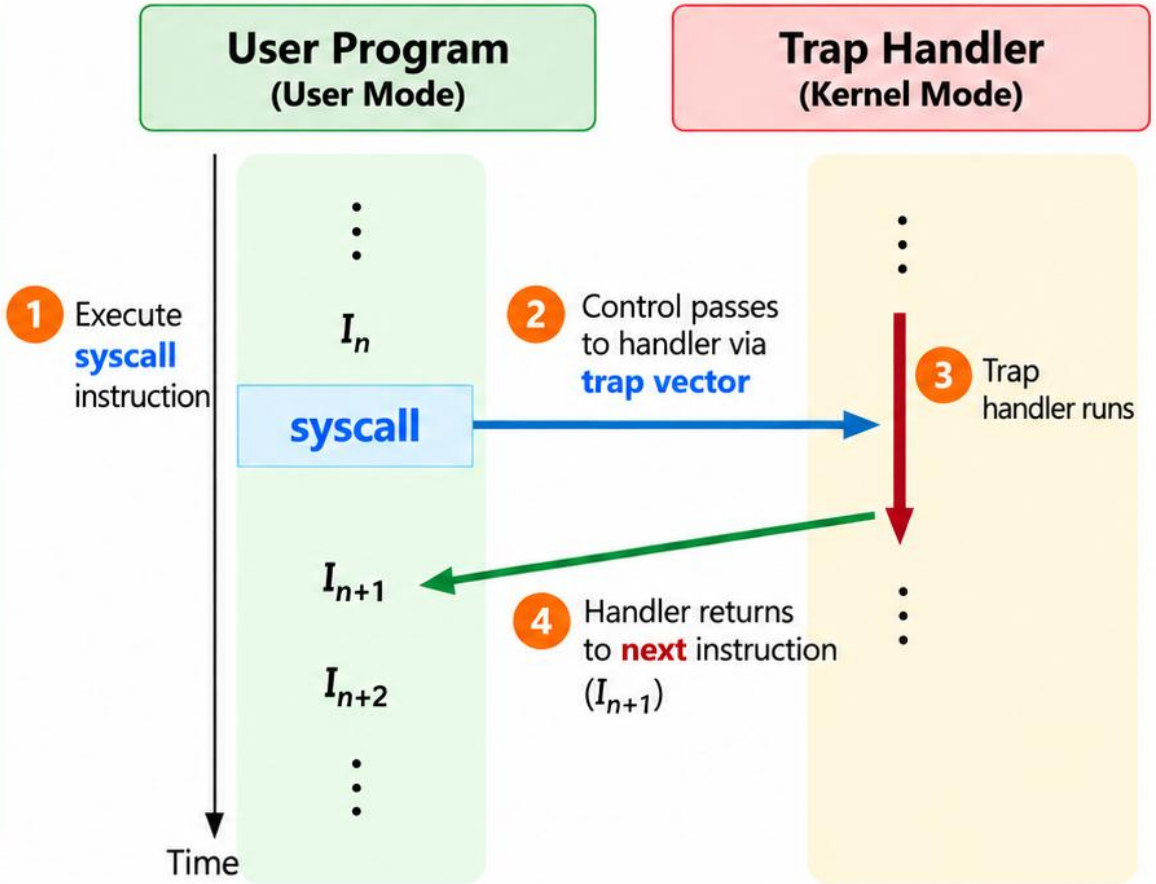
Interrupts - Asynchronous Exceptions

- Caused by events **external to the processor**
- Indicated by setting the processor's **interrupt pin**
- **Examples:** Timer interrupt, I/O interrupts from external devices
- Jump to the handler via **interrupt vector**
- Handler returns to ***"next"*** instruction



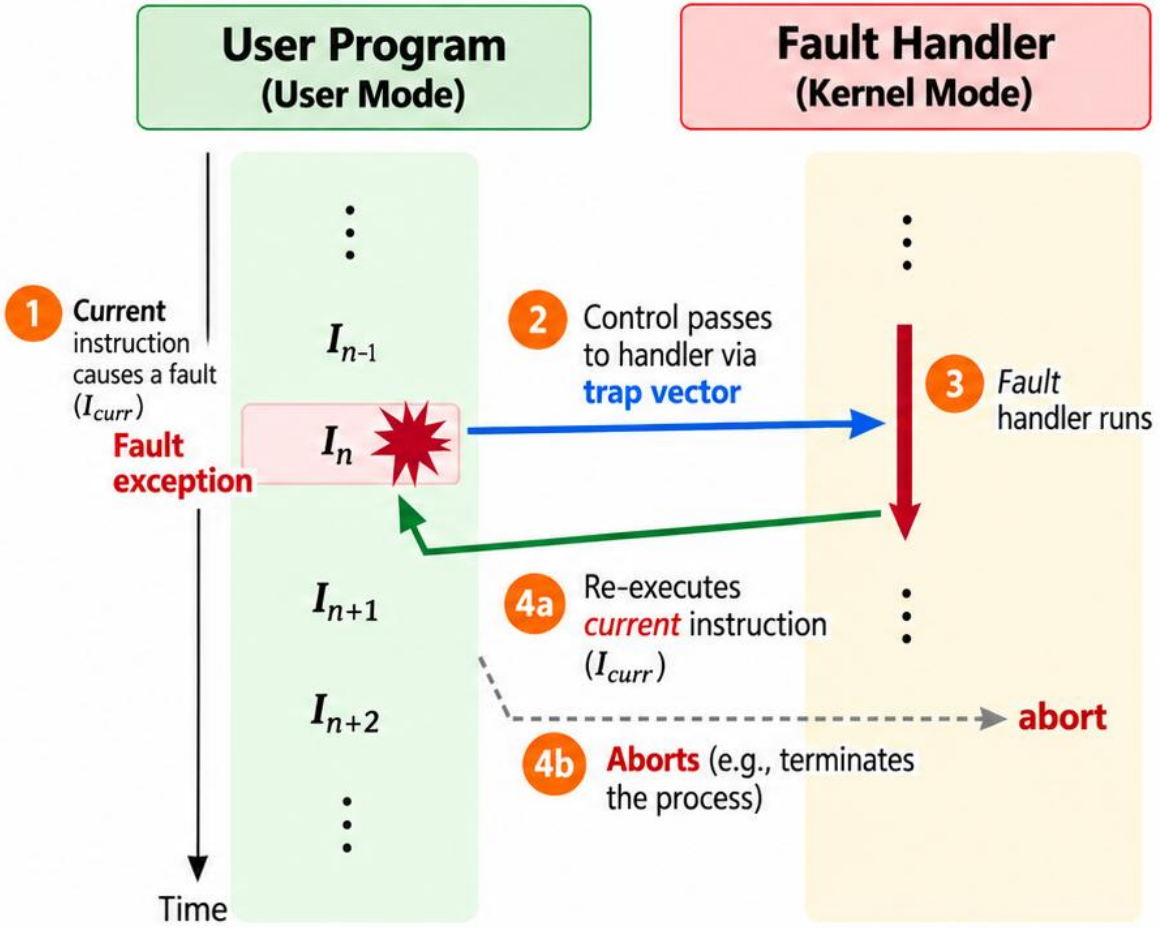
Traps - Synchronous Exceptions

- Caused by events that occur as a **result of executing an instruction**
- **Intentional**
- **Examples:** system call (e.g., syscall instruction), breakpoint traps, special instructions
- Returns control to ***"next"*** instruction



Faults - Synchronous Exceptions

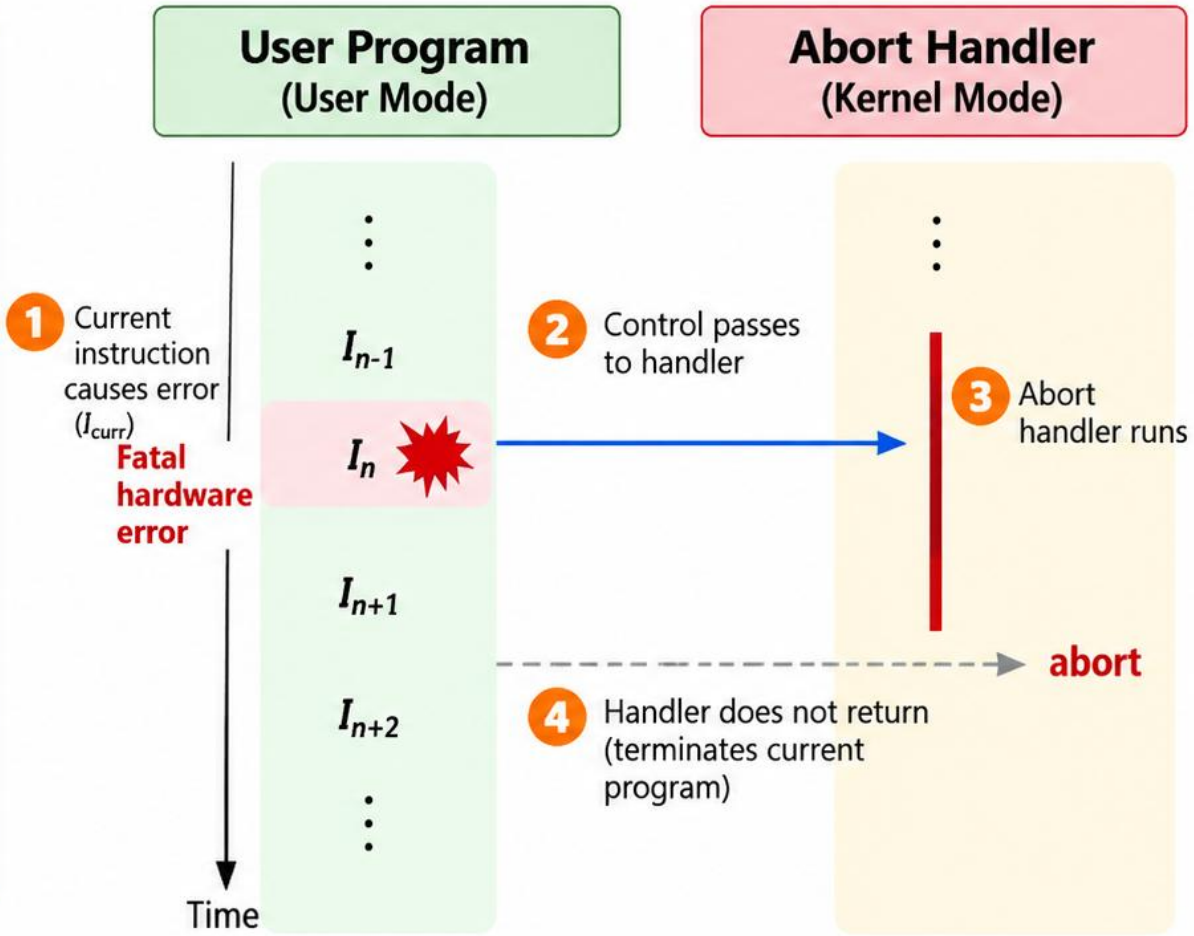
- Caused by events that occur as a **result of executing an instruction**
- **Unintentional** but possibly recoverable (recoverable or abort)
- **Examples:** page faults, protection faults (i.e., segmentation fault), floating point exceptions (i.e., divide by zero)
- Either re-executes faulting (**"current"**) instruction or **aborts**



AI-generated illustration using ChatGPT (OpenAI)

Aborts - Synchronous Exceptions

- Caused by events that occur as a **result of executing an instruction**
- **Unintentional** and unrecoverable
- **Examples:** parity error, machine check / severe hardware error
- Aborts current program



AI-generated illustration using ChatGPT (OpenAI)

Nonlocal Jump

A nonlocal jump allows control to return to a saved point in a different function, bypassing the normal call stack unwinding process.

In Linux, this is commonly provided by setjmp() / longjmp() (or siggsetjmp() / siglongjmp()).

Example: setjmp() / longjmp()

```c
#include <setjmp.h>

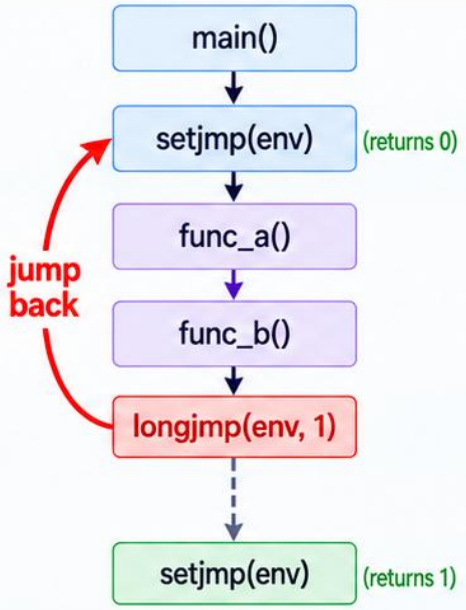
jmp_buf env;

int main() {
   if (setjmp(env) == 0) {
       func_a();
    } else {
       printf("Returned via longjmp!\n");
    }
    return 0;
}

void func_a() { func_b(); }

void func_b() { longjmp(env, 1); }
```

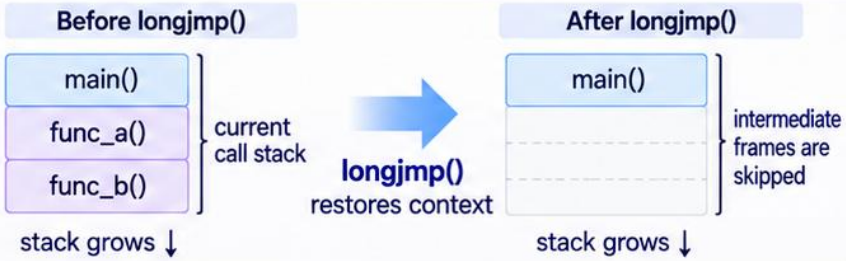
Control Flow



What Happens?

1. **setjmp() saves the current execution context**
   - register values, stack pointer, program counter, etc.
2. **longjmp() restores the saved context**
   - control jumps back to the point where setjmp() was called
   - intermediate function calls are skipped (stack is not unwound normally)

Stack View



Typical Use Cases

- **Error recovery** — e.g., parser, initialization failure
- **Resource cleanup** — e.g., without unwinding stack
- **User-space exception handling** — e.g., language runtime, JVM, etc.
- **Simpler control flow** — alternative to long error checks

Roadmap of This Week

Exceptional Control Flow (ECF) changes the normal flow of execution in response to events.
ECF can be viewed at **two levels**. This week, we focus on **Interrupts** and **Signals** in depth.

Low-Level ECF (Exception)

Events that transfer control to the kernel (hardware or CPU)

1. Interrupt



Asynchronous event from hardware

(e.g., timer, device, network)

FOCUS (this week)

Detailed handling and Linux specifics

2. Trap (System Call)



Synchronous event generated intentionally by a program

(e.g., system call)

ALREADY COVERED (Week 1)

How programs invoke the kernel

3. Fault



Synchronous event caused by an error (potentially recoverable)

(e.g., page fault)

BRIEF OVERVIEW (this week)

Main ideas and examples

4. Abort



Synchronous event caused by a serious error (cannot be recovered)

(e.g., machine check)

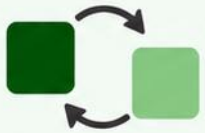
BRIEF OVERVIEW (this week)

Main ideas and examples

High-Level ECF

Mechanisms provided by the OS (software)

5. Process Context Switch



Switching the CPU from one process to another

(by scheduler)

ALREADY COVERED (Week 1)

How the OS schedules processes

6. Signal



Asynchronous notification to a process from the kernel or another process

(e.g., SIGINT, SIGSEGV)

FOCUS (this week)

Generation, delivery, and handling

7. Nonlocal Jump



Changing control flow within a process

(e.g., setjmp/longjmp, C++ exceptions)

BRIEF OVERVIEW (this week)

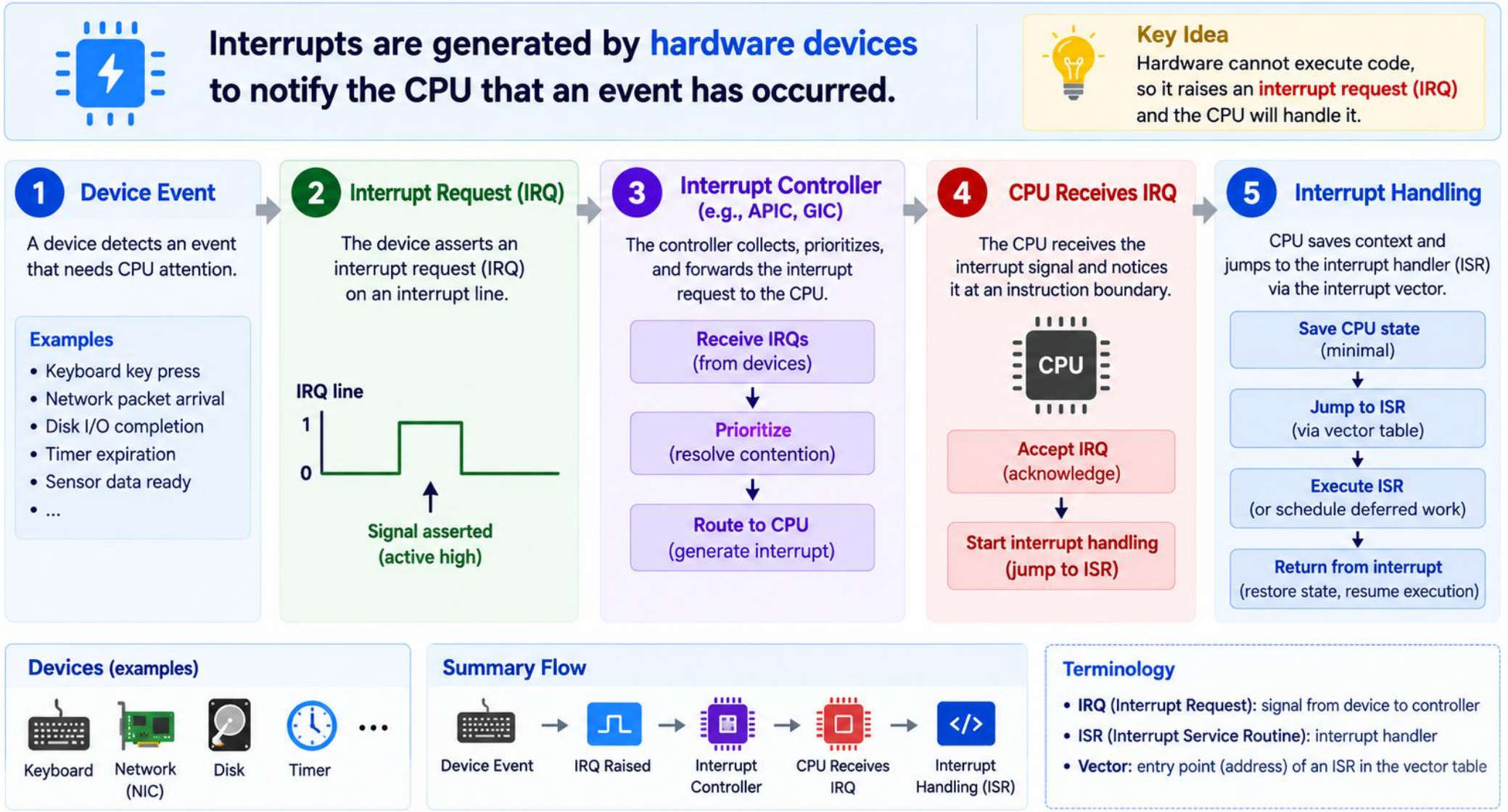
Main ideas and examples

PART 1: Linux Execution Model

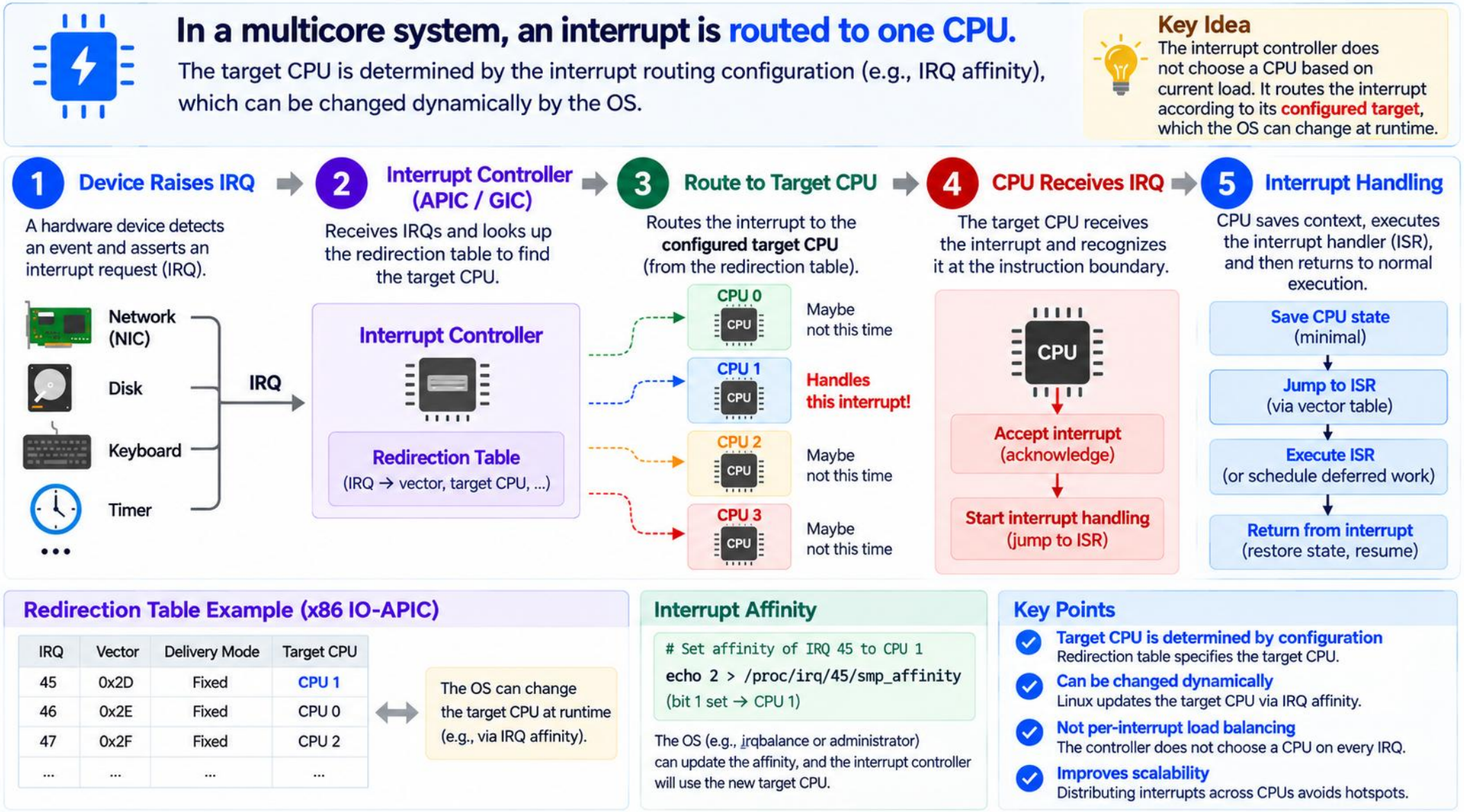
- **Week 2**

- Exceptional Control Flow (ECF)
  - How can normal control flow change?
- **Interrupts and Device Interface**
  - How does the CPU interact with hardware events?
- Signals
  - How does Linux deliver events to processes?

Interrupt Processing in General



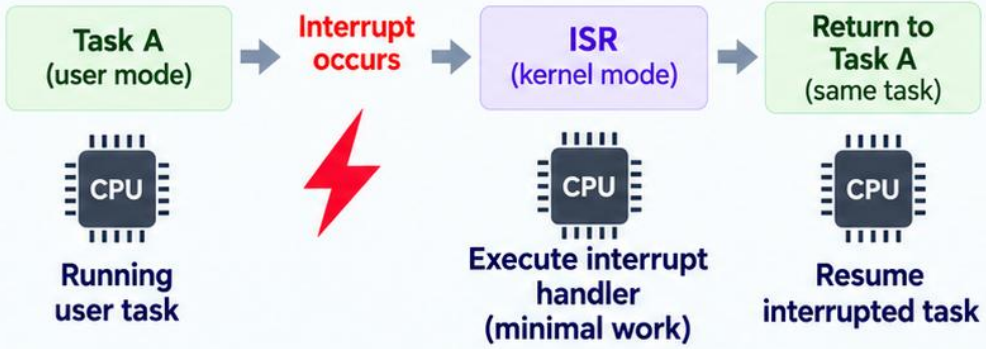
Interrupts in a Multicore System



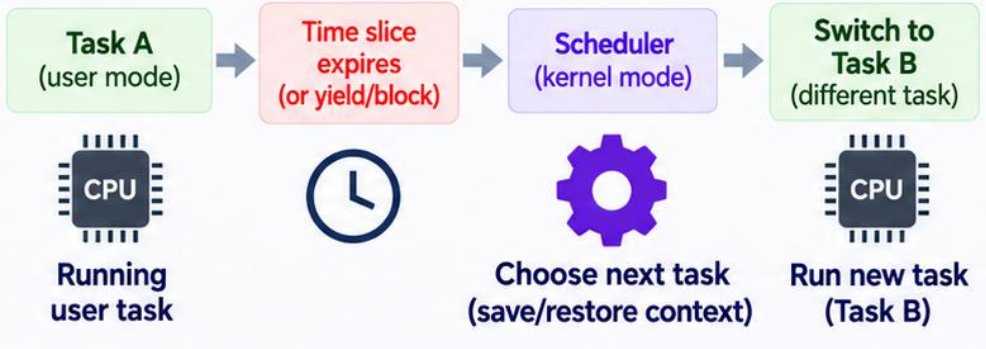
Interrupt ≠ Context Switch

An interrupt does NOT mean a context switch.
The CPU temporarily handles the event, then **returns to the interrupted task.**

1 Interrupt Handling (No Context Switch)



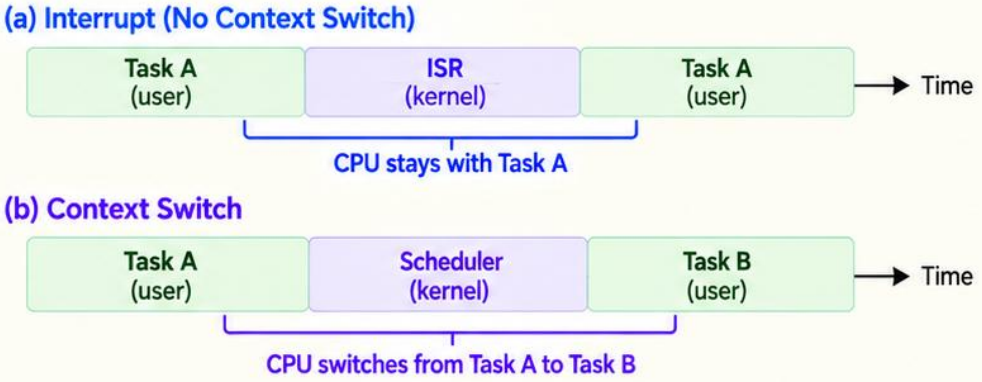
2 Context Switch (Task Switch)



3 Key Differences

Aspect	Interrupt Handling	Context Switch
Who triggers?	Hardware device (IRQ)	Scheduler (timer, block, etc.)
Purpose	Handle an external event	Run a different task
# of tasks involved	One (same task)	Two or more
Does CPU pick another task?	No	Yes
Returns to	Interrupted task	Different task
Typical latency	Very short (µs)	Longer (µs ~ ms)

4 CPU Timeline Example



Process Context vs. Interrupt Context

Process context and interrupt context are different execution contexts.
They run with different privileges, have different constraints, and are used for different purposes.

Aspect	Process Context (User or Kernel)	Interrupt Context (Kernel, ISR)
Who runs?	A process / thread (scheduled by the OS)	Interrupt handler (ISR) on behalf of a device
Where?	User mode or kernel mode (scheduled)	Kernel mode only (preempts current execution)
Can it sleep?	Yes (block, wait for I/O, etc.)	No (cannot sleep)
Can it use locks?	Yes (may sleep while waiting)	Only non-sleeping locks (spinlock, raw lock, no mutex/semaphore)
Preemptible?	May be preempted (by higher priority task/IRQ)	Not preemptible by normal tasks (keep it short)
Typical APIs	All kernel APIs allowed (e.g., read, write, malloc, mutex, copy_to_user)	Only "atomic" APIs allowed (e.g., spin_lock, kfree, copy_to_user (careful))

Process Context

- Code running as part of a process or kernel thread (scheduled by the OS).
- Can sleep and block.
- Can use blocking locks (e.g., mutex, semaphore).
- Runs in user mode or kernel mode.
- Used for normal program execution and longer tasks (e.g., system calls, file I/O).

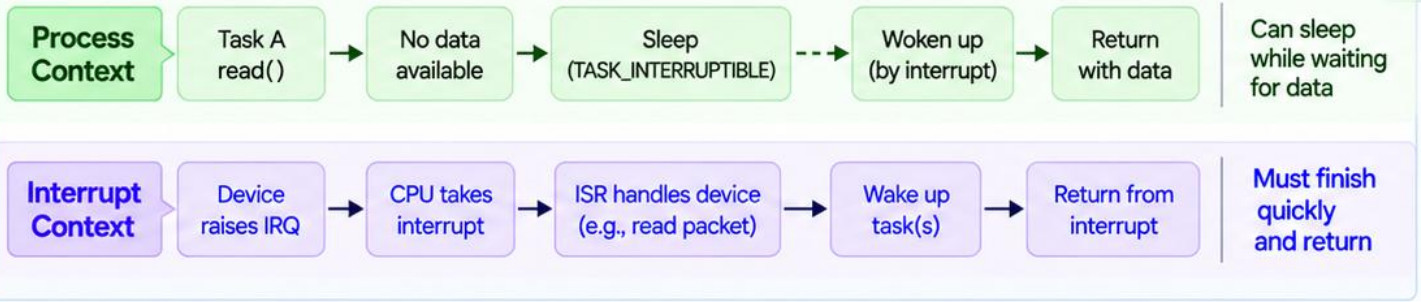


Interrupt Context

- Code running in an interrupt handler (ISR) in response to a hardware event.
- Cannot sleep or block.
- Can only use non-sleeping locks (e.g., spinlock, raw lock).
- Always runs in kernel mode.
- Preempts the current execution.
- Should be short and fast (defer longer work to bottom halves).



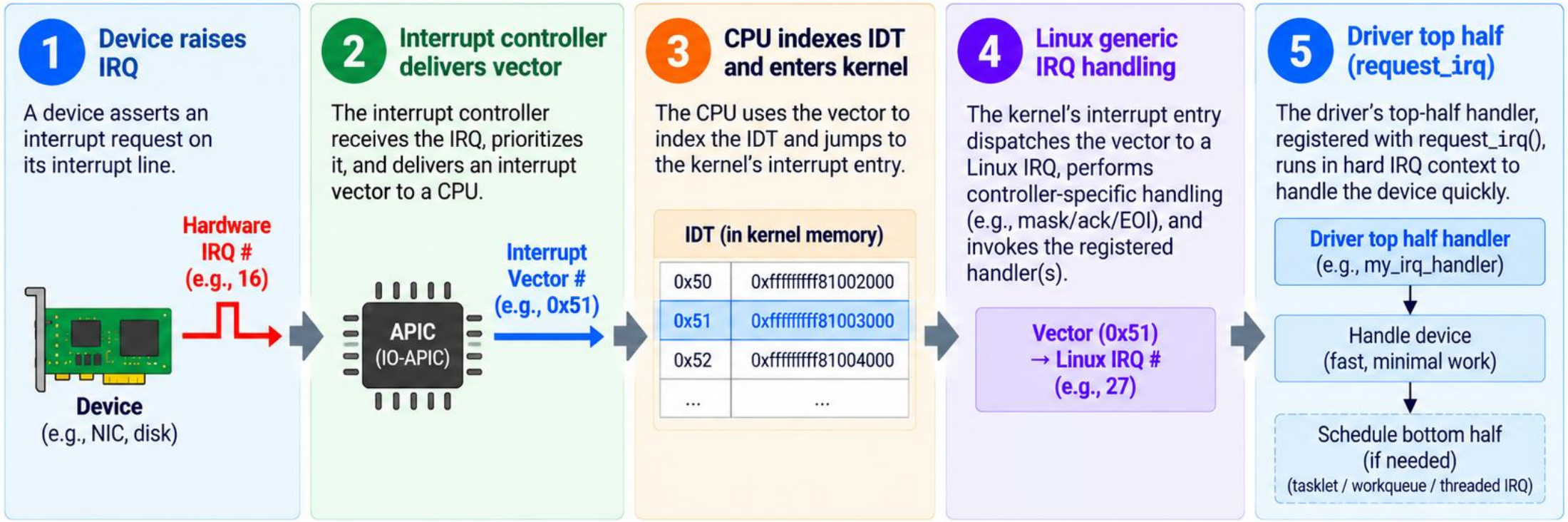
Where They Come From and an Example (read() Waiting for Data)



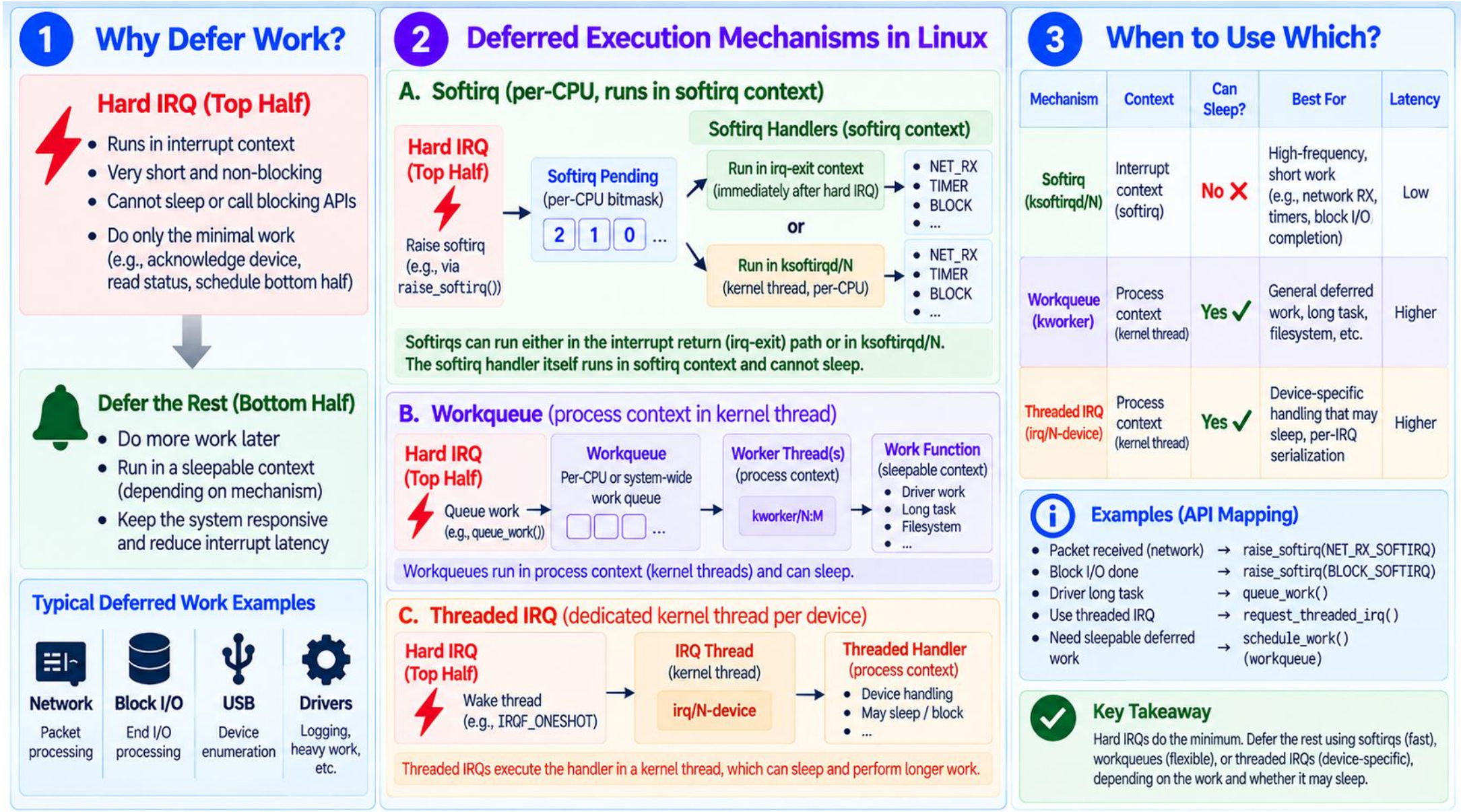
Key Idea

- Process context and interrupt context are different execution contexts with different constraints.
- Interrupt context runs with higher priority, cannot sleep, and should be short.
- Longer or blocking work should be deferred to bottom halves (softirq, tasklet, workqueue).

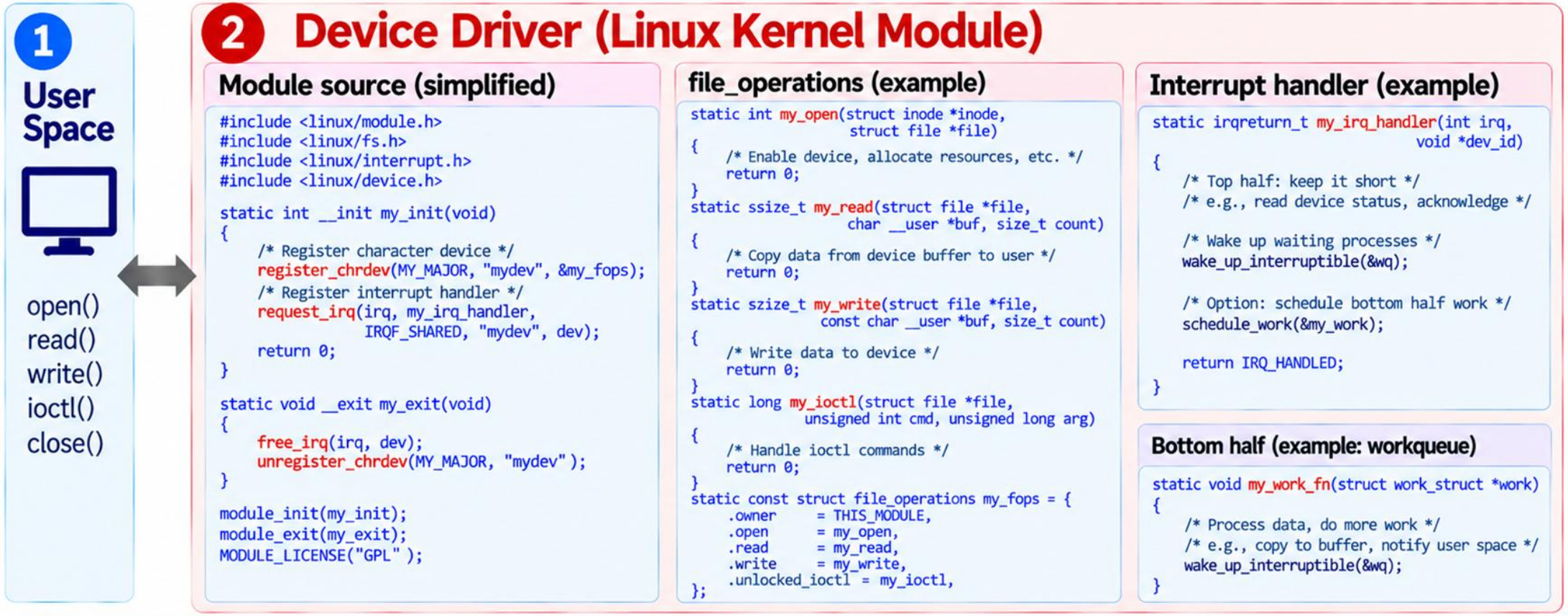
Interrupt Processing in Linux



Bottom Half Handling in Linux



Device Driver in Linux



3 Key Points

- A device driver can be **built into the kernel** or implemented as a **loadable kernel module**.
- When the module is loaded, the kernel calls my_init().
- In my_init(), the driver registers a character device using **register_chrdev()** and typically registers the interrupt handler using **request_irq()**.
- The file_operations structure contains the functions invoked by system calls from user space (open, read, write, ioctl, close).
- **my_irq_handler()** runs when a hardware interrupt occurs, processes the device event, and typically wakes up waiting processes or schedules a bottom half (workqueue, threaded IRQ, etc.).
- Keep the top half short and defer longer work to a bottom half.
- The device node (/dev/mydev) can be created in two ways:
  - Manually using **mknod** (e.g., mknod /dev/mydev c <major> 0)
  - Automatically by the driver during registration (e.g., using **class_create()** and **device_create()**).

Example: Interrupt and Blocking I/O

1 Register the interrupt handler (module initialization)

This code is part of a loadable kernel module (character device driver).
Load the driver into the kernel with: `insmod mydev.ko`

```
static int __init mydev_init(void)
{
    int ret;

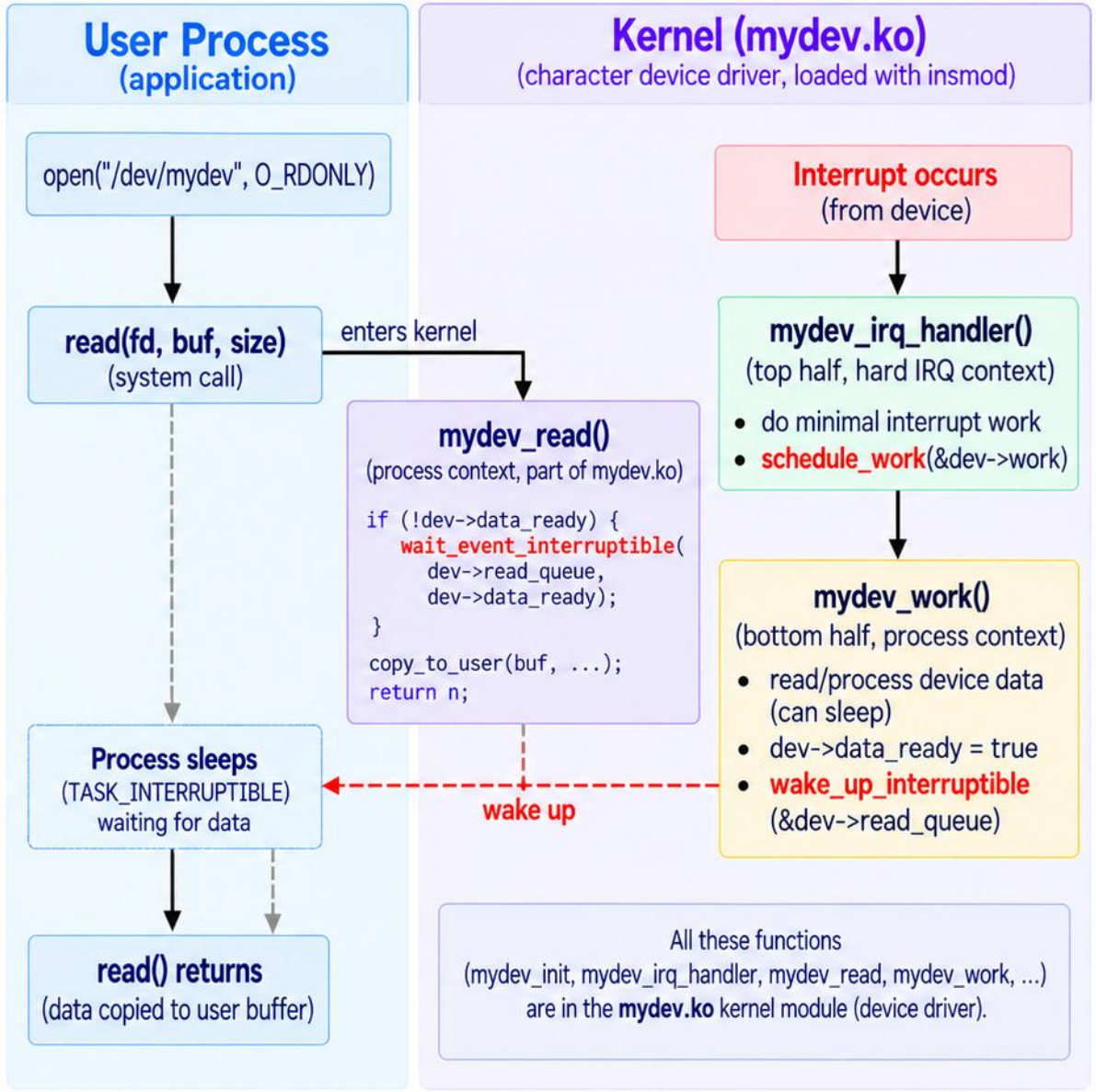
    ret = request_irq(dev->irq,
                      mydev_irq_handler,  /* IRQ number */
                      IRQF_SHARED,        /* top half */
                      "mydev",            /* device name */
                      dev);               /* device data */
    if (ret)
        return ret;
    return 0;
}
```

2 Top half: interrupt handler (hard IRQ context)

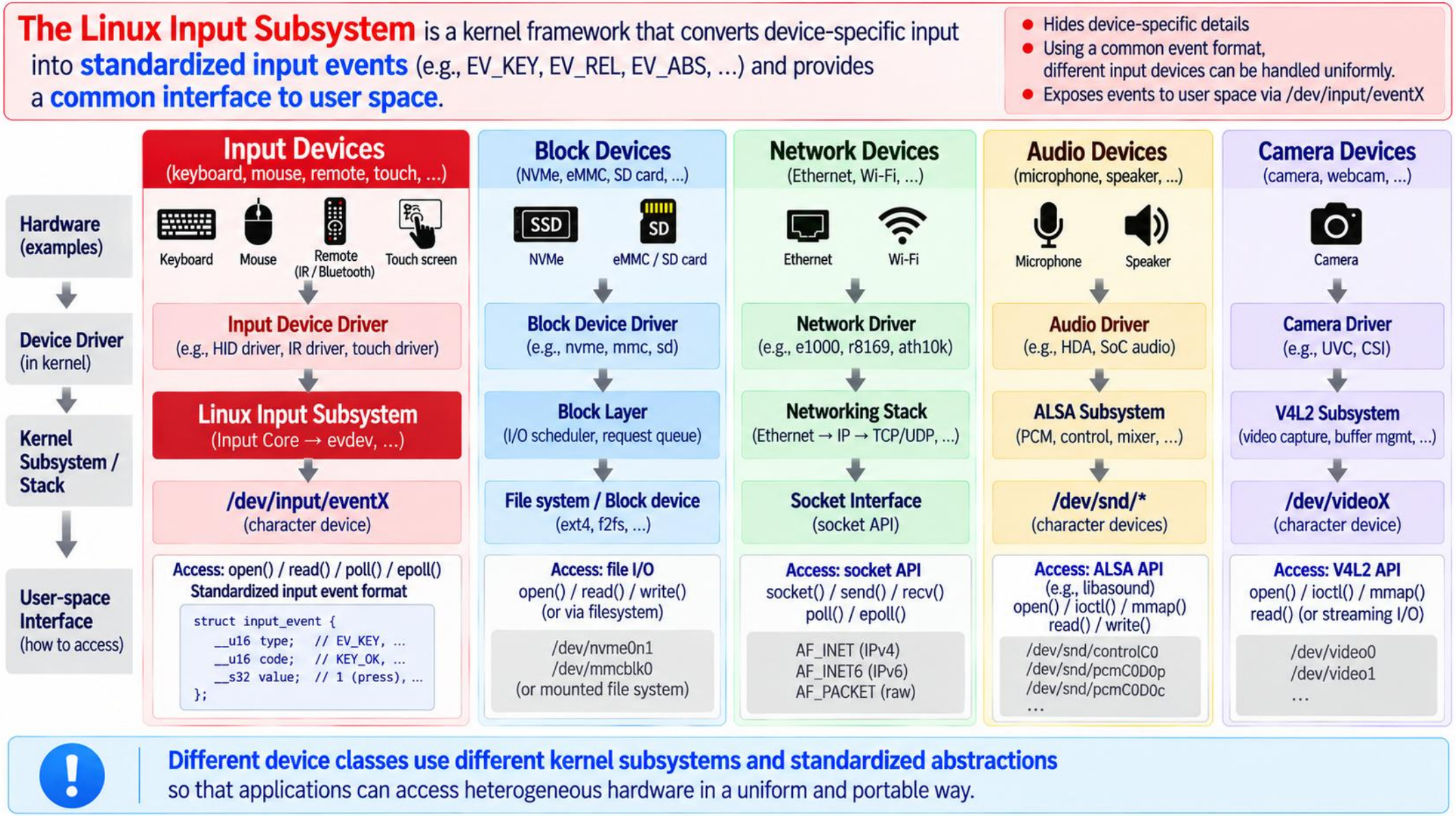
```
static irqreturn_t mydev_irq_handler(int irq, void *data)
{
    struct mydev *dev = data;
    /* Do minimal interrupt work (e.g., read status, clear irq). */
    /* Keep it short and non-blocking. */
    schedule_work(&dev->work); /* defer the rest to bottom half */
    return IRQ_HANDLED;
}
```

3 Bottom half: deferred work (process context)

```
static void mydev_work(struct work_struct *work)
{
    struct mydev *dev = container_of(work, struct mydev, work);
    /* Process device data (can sleep). */
    mydev_read_data(dev);
    dev->data_ready = true;
    /* Wake up processes waiting in read() */
    wake_up_interruptible(&dev->read_queue);
}
```



What is the Linux Input Subsystem?



How the Linux Input Subsystem Works

1 Device Registration Path (when an input device is initialized/connected)

Bus / Driver Framework

- Detect / enumerate device (e.g., USB, I²C, SPI, PCI)
- Match with a driver (of_device_id, etc.)
- Call driver's probe()

Input Device Driver (probe)

DRIVER

- Allocate and initialize `struct input_dev`
- Set device capabilities (EV_KEY, EV_REL, etc.)
- Register the device with the input core

```
static int my_probe(...) {
  input_register_device(dev);
  return 0;
}
```

Input Core (kernel framework)

- Maintain list of input devices
- Handle registration and lifetime management
- Provide interface for input drivers
- Pass input events to appropriate handler(s) (e.g., evdev)

evdev (input handler)

- Called by input core when a new input device is registered
- Create and register a character device
- Implement file operations (open/read/poll, etc.)
- Receive input events and provide them to user space

/dev/input/eventX (character device)

/dev

- Device node created by devtmpfs (in the kernel)
- One eventX device per input device

```
/dev/input/event0
/dev/input/event1
...
```

User Space (Application)

- Open the device node and read input events (when needed)

```
int fd = open(
  "/dev/input/event0",
  O_RDONLY);
struct input_event ev;
read(fd, &ev, sizeof(ev));
```

2 Input Event Path (when a button/key is pressed)

Hardware Event

- Key/button press, mouse movement, etc.
- Device generates an interrupt (IRQ)

Input Device Driver (interrupt handler)

- **Top half** (IRQ handler)
  - Read device status
  - Acknowledge interrupt
  - Schedule bottom half
- **Bottom half** (workqueue)
  - Convert to input events
  - Report events to input core)

Input Core

- Receive input events from the driver
- Process and dispatch to registered handler(s) (e.g., evdev)

evdev (input handler)

- Receive input events
- Store events in per-client event buffer
- Wake up processes waiting for input events (wake_up_interruptible())

/dev/input/eventX (character device)

/dev

- Buffered input events can be read by user space (read/poll/epoll)

User Space (Application)

- Read input events

```
struct input_event ev;
read(fd, &ev, sizeof(ev));
```

- Use higher-level libraries (optional)
  - **libevdev** (low-level helper)
  - **libinput** (higher-level API)

The device node `/dev/input/eventX` is created by **devtmpfs** (in the kernel). It is not created by the driver or udev. (udev may set permissions, ownership, or create symlinks.)

What Happens When You Press a Button on a webOS TV?

1 User Input

TV Remote Control



Button Press (e.g., OK)

(IR / Bluetooth / / USB)

2 Hardware Interrupt

Device Controller
(IR / USB / Bluetooth)

Generates hardware interrupt (IRQ)

APIC / GIC
(Interrupt Controller)

Invoke driver's IRQ handler

3 Device Driver (hardware-specific)

Remote / Input Driver
(e.g., IR driver, USB HID driver)

- Handle IRQ in interrupt context
- Read device registers / data (e.g., scan code 0x35)
- Decode to Linux input event (e.g., KEY_OK)
- Report the event to input core

```
input_report_key(dev, KEY_OK, 1);
input_sync(dev);
```

4 Linux Input Subsystem (input core + evdev)

Input Core
(kernel framework)

- Receive the input event (EV_KEY, KEY_OK, 1)
- Dispatch to connected handler(s) (e.g., evdev->event())

evdev (input handler)

- evdev->event() is called by the input core
- Store the event in a buffer (ring buffer)
- Wake up processes waiting for events

/dev/input/eventX
(character device)

- Provides the interface to user space
- read() / poll() / epoll()

5 LSM (Surface Compositor + Input Handling)

Receive key event
(from input subsystem)

Identify the key
(e.g., OK / HOME / BACK)

Decide how to handle it

- Handle as system action (e.g., HOME, menu)

or

- Send to active application (e.g., OK, BACK)

System UI action / Forward to application

Key Points

1. User presses a button on the remote control.
2. Hardware generates an interrupt (IRQ), and the driver's IRQ handler runs.
3. The driver reads device data, decodes it to a Linux input event, and reports it to the input core.
4. The input core dispatches the event to evdev, which buffers it and exposes it via /dev/input/eventX.
5. LSM receives the key event, identifies the key, and either handles it as a system action or sends it to the active app.

Seeing Deferred Work in Linux

1 Find kernel threads with ps

```
$ ps -eo pid,psr,stat,comm | grep -E 'ksoftirqd|kworker|irq/'
```

PID	PSR	STAT	COMMAND
21	0	S	ksoftirqd/0
22	1	S	ksoftirqd/1
45	0	I	kworker/0:1-events
67	2	I	kworker/2:0-mm_percpu_wq
1578	0	S	irq/16-i2c_hid
1601	1	S	irq/25-eth0

pid
Process ID (thread ID for kernel threads)

psr
CPU currently running on (or last CPU)

stat
Task state (R: running, S: sleeping, I: idle, ...)

comm
Task/Thread name (kernel thread)

2 What are these kernel threads?

ksoftirqd/N	Kernel thread that executes pending softirqs on CPU N (softirq context).	Can sleep? **No** (softirq handler must not sleep)
kworker/X:Y	Kernel worker thread from workqueue (process context).	Can sleep? **Yes** (general deferred work)
irq/N-device	Threaded IRQ handler for device N (per-IRQ kernel thread).	Can sleep? **Yes** (device handling, may sleep)

3 Pattern used in grep

```
grep -E 'ksoftirqd|kworker|irq/'
```

- **-E** : use extended regular expression ("|" means OR)
- **ksoftirqd** : matches softirq threads
- **kworker** : matches workqueue threads
- **irq/** : matches threaded IRQ threads (e.g., irq/16-i2c_hid)

The '/' is just a literal character in the thread name (e.g., irq/16-eth0). It is not a special regex symbol.

Key Points

- Use **ps** to see kernel threads handling deferred work.
- **psr** shows the CPU where the task is running.
- **ksoftirqd** runs softirqs (cannot sleep), while **kworker** and **irq/** threads can sleep.

Interrupt Counters in Linux

1 View Interrupt Counters with /proc/interrupts

/proc/interrupts shows, for each IRQ, how many times each CPU has handled the interrupt, which controller/type it is, and the device/driver name.

```
$ cat /proc/interrupts | head -n 6
```

IRQ	CPU0	CPU1	CPU2	CPU3	Controller/Type	Device
0:	12345	10234	9987	8876	IO-APIC 2-edge	timer
1:	5678	4321	3987	3456	IO-APIC 1-edge	keyboard
9:	0	0	1	0	IO-APIC 1-edge	acpi
16:	23456	21001	20543	19876	IO-APIC 16-edge	i8042
25:	34567	32011	31022	29876	PCI-MSI	eth0-TxRx-0
...						

IRQ number | Per-CPU interrupt counts (how many times handled) | Interrupt controller and type | Device / driver

2 Column Explanation

Column	Meaning
IRQ	Linux IRQ number (virtual interrupt number)
CPU0...N	How many times each CPU has handled this IRQ
Controller/Type	Interrupt controller and trigger type (e.g., IO-APIC, PCI-MSI, edge/level)
Device	Device or driver name that handles this interrupt (e.g., timer, keyboard, i8042, eth0)

3 When to Use This Information

- **Identify high-rate interrupt sources** (e.g., network, storage, hardware issues)
- **Check load distribution across CPUs** (validate IRQ affinity, detect imbalance)
- **Verify device/driver behavior** (is the device generating interrupts as expected?)

4 Useful Commands

Show all interrupts

```
$ cat /proc/interrupts
```

All IRQs and per-CPU counts

Watch in real time

```
$ watch -n 1 'cat /proc/interrupts'
```

See how counts change

Filter for a specific device

```
$ grep -i nvme /proc/interrupts
```

Show only NVMe-related IRQs

5 Key Takeaways

- **/proc/interrupts** quickly shows **which interrupts occur, on which CPUs,** and how often.
- The **Controller/Type field is controller-specific** (e.g., IO-APIC, PCI-MSI).
- Use it together with other tools (top, mpstat, perf) to understand performance impact.

Tracing Interrupt Processing in Linux

1 Observe IRQ Events

Use perf tracepoints to see **which interrupts occur, when, and on which CPU.**

```
$ sudo perf record -a \
    -e irq:irq_handler_entry \
    -e irq:irq_handler_exit \
    -- sleep 10
```

Record IRQ handler entry/exit events using tracepoints.

Example output (perf script)

TIME	CPU	EVENT	IRQ	HANDLER
10.123456	0	irq_handler_entry	16	nvme_irq
10.123789	0	irq_handler_exit	16	nvme_irq
10.224561	3	irq_handler_entry	1	i8042_interrupt
10.224890	3	irq_handler_exit	1	i8042_interrupt
...				

What this shows

- Which IRQs occur, when, and on which CPU
- How often each interrupt handler is invoked

2 Find CPU Hotspots (Where Time Is Spent)

Use system-wide profiling to find **where the CPU spends time** (including interrupt handlers).

```
$ sudo perf record -a -g -- sleep 10
$ sudo perf report
```

Profile CPU usage (with call stacks) and find the hotspots.

Example output (perf report)

Overhead	Command	Shared Object	Symbol
34.12%	[kernel]	[k]	nvme_interrupt
12.57%	[kernel]	[k]	blk_mq_complete_request
8.21%	[kernel]	[k]	__hrtimer_run_queues
5.03%	[kernel]	[k]	update_process_times
3.76%	[kernel]	[k]	tick_sched_timer
2.91%	[kernel]	[k]	tcp_clean_rtx_queue
...			

What this shows

- Which functions consume the most CPU time (inclusive)
- Interrupt handlers can appear here if they use significant CPU time

Key Takeaway

- Use **tracepoints** (irq:irq_handler_entry/exit) to see **IRQ events**.
- **Use profiling** (perf record -g) to find where the CPU time is actually spent.

IRQ tracepoints show events.
perf profiling shows CPU hotspots.

Find Where the Event is Lost in Linux

1 Check Hardware & IRQ

Did the interrupt occur?



SoC / Interrupt Controller

Button press (e.g., OK)

```
$ cat /proc/interrupts
IRQ   CPU0  CPU1  ... Device
123:  157    0    ... ir-receiver
...
```

Press the button and check that the IRQ count increases.

2 Trace Kernel Path

What happens inside the kernel?

Input Driver (IRQ handler / workqueue)

↓

Linux Input Subsystem (input_report_key)

↓

evdev (event buffering, wake up)

```
$ sudo trace-cmd record \
   -e input -e irq -e sched
# reproduce the button press
$ sudo trace-cmd report | less
```

Trace shows the input event flow in the kernel (event names depend on kernel version).

3 Observe evdev Events

What events reach user space?

/dev/input/eventX (character device)

```
$ ls -l /dev/input/
crw-rw---- 1 root input  13, 64 event0
crw-rw---- 1 root input  13, 65 event1
crw-rw---- 1 root input  13, 66 event2
...
```

```
$ sudo evtest /dev/input/event2
Input device: IR Remote Control
Event: time 1758001234.123456,
  type 1 (EV_KEY), code 352 (KEY_OK), value 1
Event: time 1758001234.123789,
  type 0 (EV_SYN), code 0 (SYN_REPORT), value 0
Event: time 1758001234.567890,
  type 1 (EV_KEY), code 352 (KEY_OK), value 0
Event: time 1758001234.568123,
  type 0 (EV_SYN), code 0 (SYN_REPORT), value 0
...
```

Shows the actual struct input_event data delivered to user space.

4 Check User Space

Does LSM or the application receive and handle it?

LSM (Surface Compositor + Input Handling)

↓

Application (e.g., Home Launcher, app)

1) LSM (systemd service)

```
$ sudo journalctl -u lsm -f
... Received KEY_OK
... Launching app ...
```

2) General application (not a systemd service)

- Run in terminal → see stdout/stderr

```
$ ./myapp
Received KEY_OK
Launching app ...
```

- Attach with strace

```
$ sudo strace -e read -tt -p <pid>
```

- Check application log file (e.g., /var/log/..., platform log)

End-to-end debugging: Find where the event is lost

If a button does not work, check each layer in order:

IRQ count → Driver → Input subsystem → evdev → LSM → Application

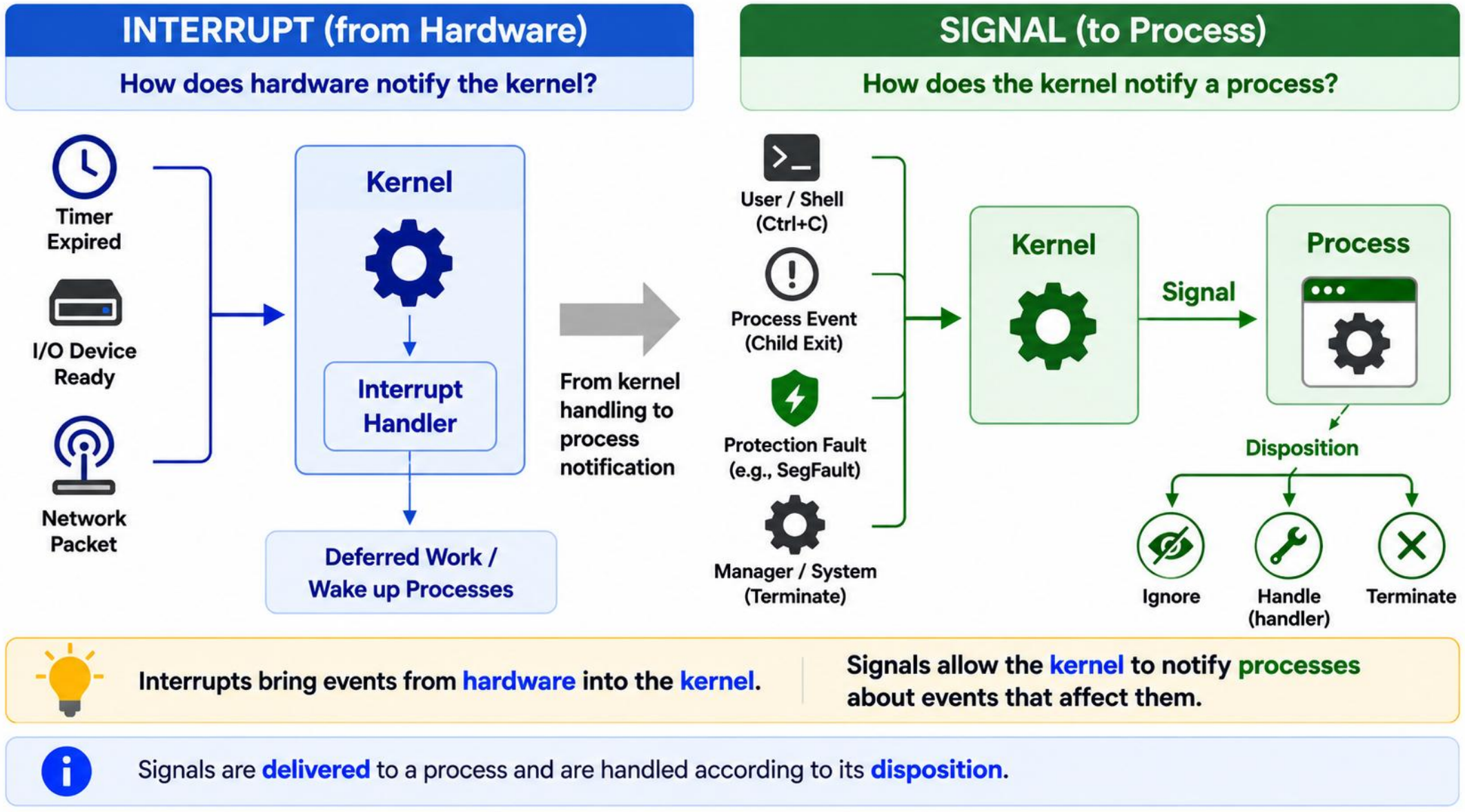
- ✓ IRQ count increases in /proc/interrupts
- ✓ Kernel trace shows input event
- ✓ evtest shows EV_KEY / KEY_xxx
- ✓ LSM logs show the event (journalctl)
- ✓ Application logs or stdout show the event

PART 1: Linux Execution Model

- **Week 2**

- **Exceptional Control Flow (ECF)**
  - How can normal control flow change?
- **Interrupts and Device Interface**
  - How does the CPU interact with hardware events?
- **Signals**
  - How does Linux deliver events to processes?

How Does Kernel Notify a Process? – Signals



Signal Concepts: Generating a Signal

1

A signal can be generated in two main ways:

A. By the kernel (system event)

- **CPU exception** (e.g., divide by zero → SIGFPE)
- **Timer event** (e.g., real-time timer → SIGALRM)
- **Child process terminates** (→ SIGCHLD)
- **Terminal input** (e.g., Ctrl-C → SIGINT)
- ...

B. By another process (explicit request)

- `kill(pid, SIGINT)` (request to send a signal)
- ...

2

When a signal is generated, the kernel identifies the target process (or thread) and updates its signal state by **marking the signal as pending**.

2-1 Find the target (process or thread)

- From the event context (e.g., current process)
- or from a system call (e.g., kill)

2-2 Check permission and validity

- Is the signal number valid?
- Is the sender allowed to send it?

2-3 Set the pending bit

- Mark the signal as pending in the kernel-maintained pending signal set (for the target process or thread).

3

Example: generating SIGINT (signal 2)

The kernel sets the bit for signal 2 in the target's pending signal set.

Before: pending signal set

0	1	2	3	4	5	6	7	···	N-1
0	0	0	0	0	0	0	0	···	0

SIGINT (signal 2)

After: pending signal set

0	1	2	3	5	6	7	8	···	N-1
0	0	1	0	0	0	0	0	···	0

SIGINT (signal 2)

(For standard signals, the pending state is a single bit. If the bit is already 1, it remains 1.)

Key Takeaways

- Generating a signal means the kernel **records the signal** in the target's signal state (by **setting a pending bit**).
- It **does NOT** mean that the signal handler runs immediately.
- Signals can be generated by kernel events or by other processes (e.g., kill).
- Signal delivery happens later, when the signal is unblocked and the kernel is about to return to user mode (see next slide).

Signal Concepts: Blocking / Pending a Signal

1 Each thread has a signal mask

- Each thread has **its own** signal mask (blocked set).
- A blocked signal is not delivered until it is unblocked.

Thread A: signal mask (blocked set)

0	1	2	3	4	5	6	7	⋯	N-1
0	0	0	0	0	0	0	0	⋯	0

Initially, no signals are blocked.

Block SIGINT (signal 2)

Thread A: signal mask (blocked set)

0	1	2	3	4	5	6	7	⋯	N-1
0	0	1	0	0	0	0	0	⋯	0

SIGINT (signal 2) is blocked for this thread

2 Blocking and unblocking signals

A. sigprocmask() example (single-threaded)

```
#include <signal.h>

sigset_t set;

sigemptyset(&set);         // start with empty set
sigaddset(&set, SIGINT);   // block SIGINT (signal 2)

if (sigprocmask(SIG_BLOCK, &set, NULL) == -1) {
    perror("sigprocmask");
}
```

B. Using pthread_sigmask() in multi-threaded programs

```
#include <signal.h>
#include <pthread.h>

pthread_sigmask(SIG_BLOCK, &set, NULL);   // block
pthread_sigmask(SIG_UNBLOCK, &set, NULL); // unblock
```

- **sigprocmask()** is used in single-threaded programs.
- **pthread_sigmask()** should be used in multi-threaded programs (affects only the calling thread).

3 Effect of blocking a signal

Before blocking / no pending signal

	0	1	2	3	4	5	6	7	⋯	N-1
Blocked	0	0	0	0	0	0	0	0	⋯	0
Pending	0	0	0	0	0	0	0	0	⋯	0

Block SIGINT (signal 2)

	0	1	2	3	4	5	6	7	⋯	N-1
Blocked	0	0	1	0	0	0	0	0	⋯	0
Pending	0	0	0	0	0	0	0	0	⋯	0

Generate SIGINT while blocked

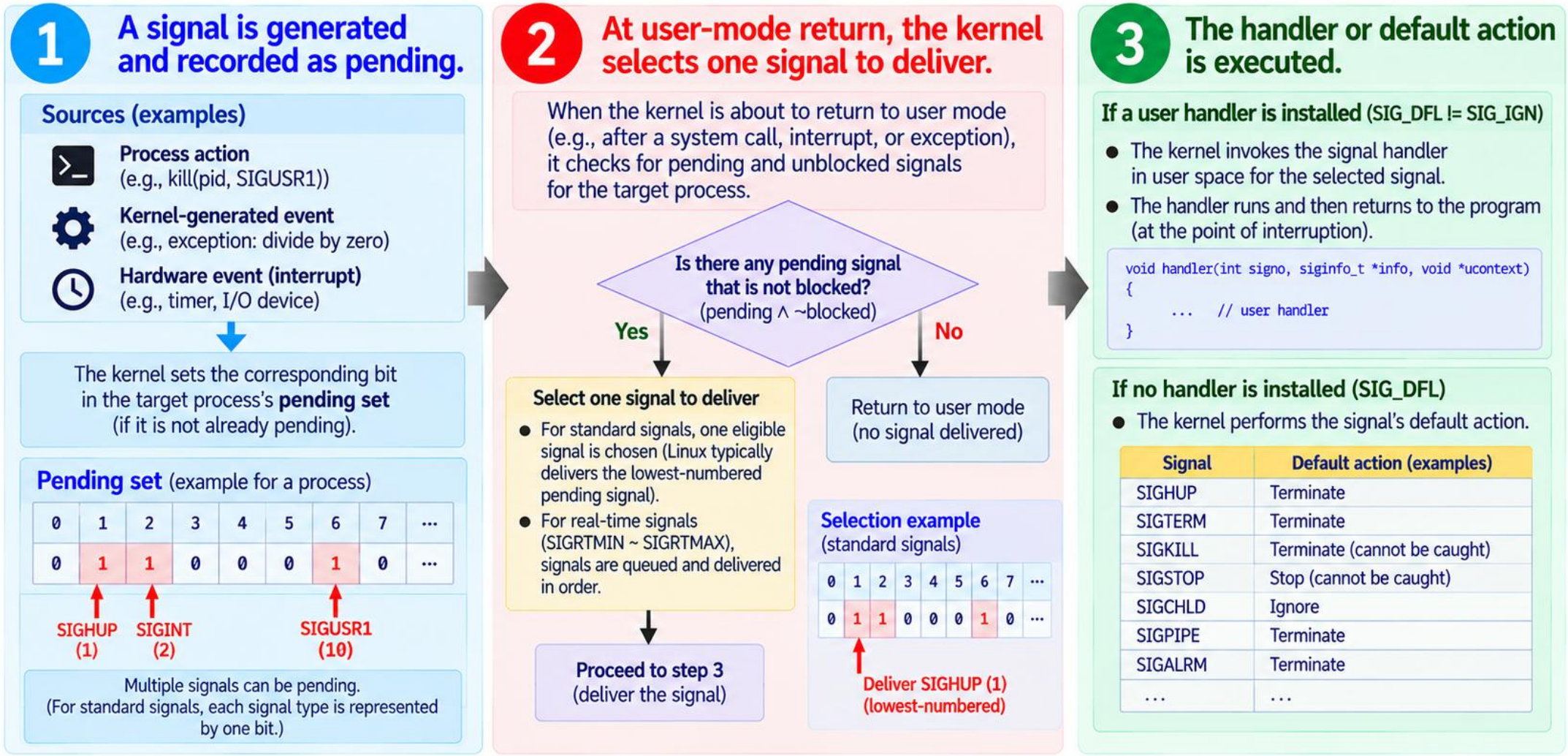
	0	1	2	3	4	5	6	7	⋯	N-1
Blocked	0	0	1	0	0	0	0	0	⋯	0
Pending	0	0	1	0	0	0	0	0	⋯	0

SIGINT is pending (not delivered yet)

Key Points

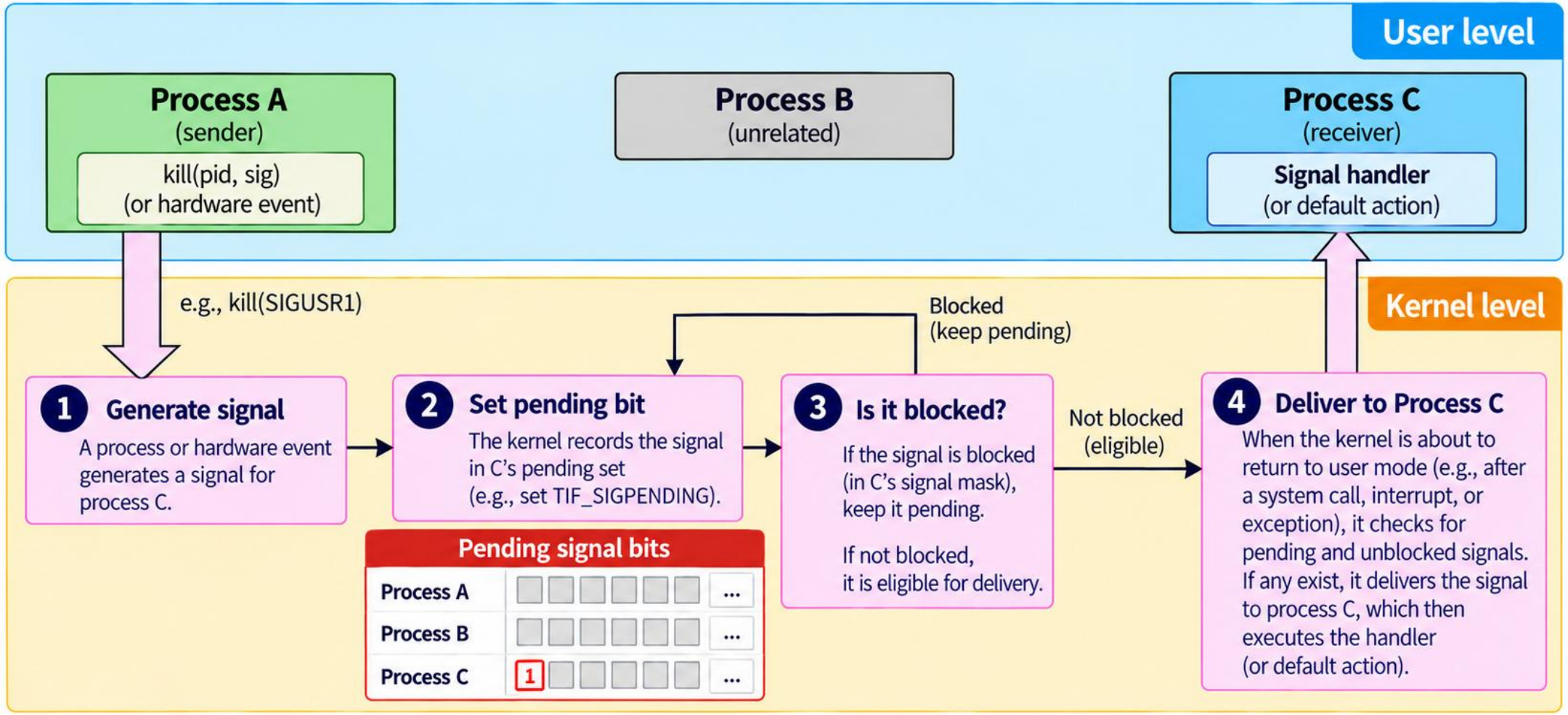
- Each thread has its own signal mask (blocked set).
- A blocked signal is marked as pending if it is generated, but it is **not delivered** until it is unblocked.
- Use `sigprocmask()` in single-threaded programs and `pthread_sigmask()` in multi-threaded programs (affects only the calling thread).
- Blocking a signal does not discard it; it only delays delivery.
- **Signal dispositions** (handlers/ignore/default) are **process-wide**, while **signal masks** are **per-thread**.

Signal Concepts: Delivering a Signal



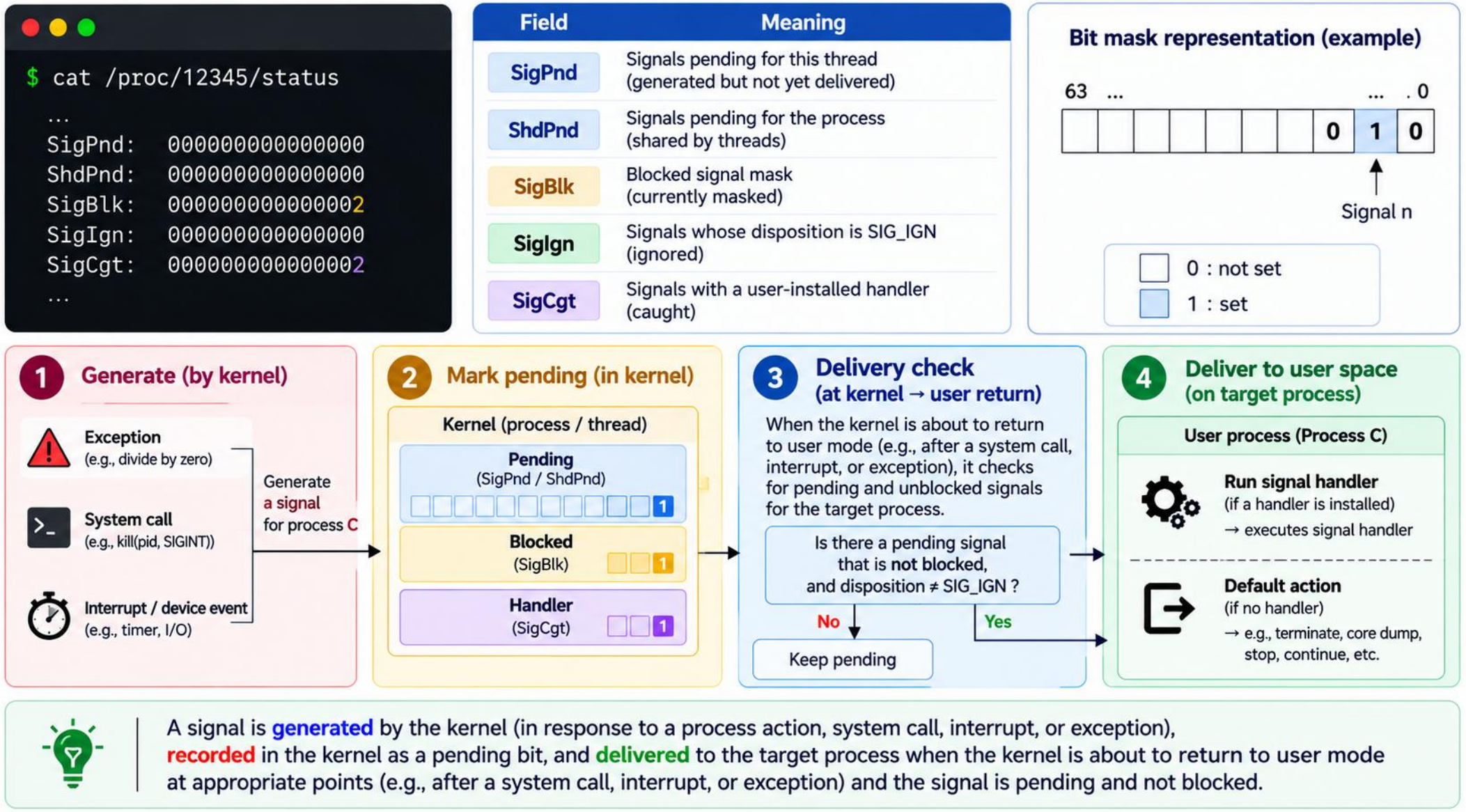
A signal is **generated** by a process or hardware event, **recorded** in the kernel as a pending bit, and **delivered** to the target process when the kernel is about to return to user mode (e.g., after a system call, interrupt, or exception) and the signal is pending and not blocked.

Altogether: Generating / Pending / Delivering



A signal is **generated** by a process or hardware event, **recorded** in the kernel as a pending bit, and **delivered** to the target process when the kernel is about to return to user mode (e.g., after a system call, interrupt, or exception) and the signal is pending and not blocked; the process then executes the handler (or default action).

Signal States in Linux



Generating Signals: Process Groups

1 A job is the set of processes created in a single command line

- Processes connected by pipes (|) belong to the **same job**
- Processes separated by ; or & are **different jobs**

Examples

```
$ cat access.log | grep ERROR | sort > result.txt
```

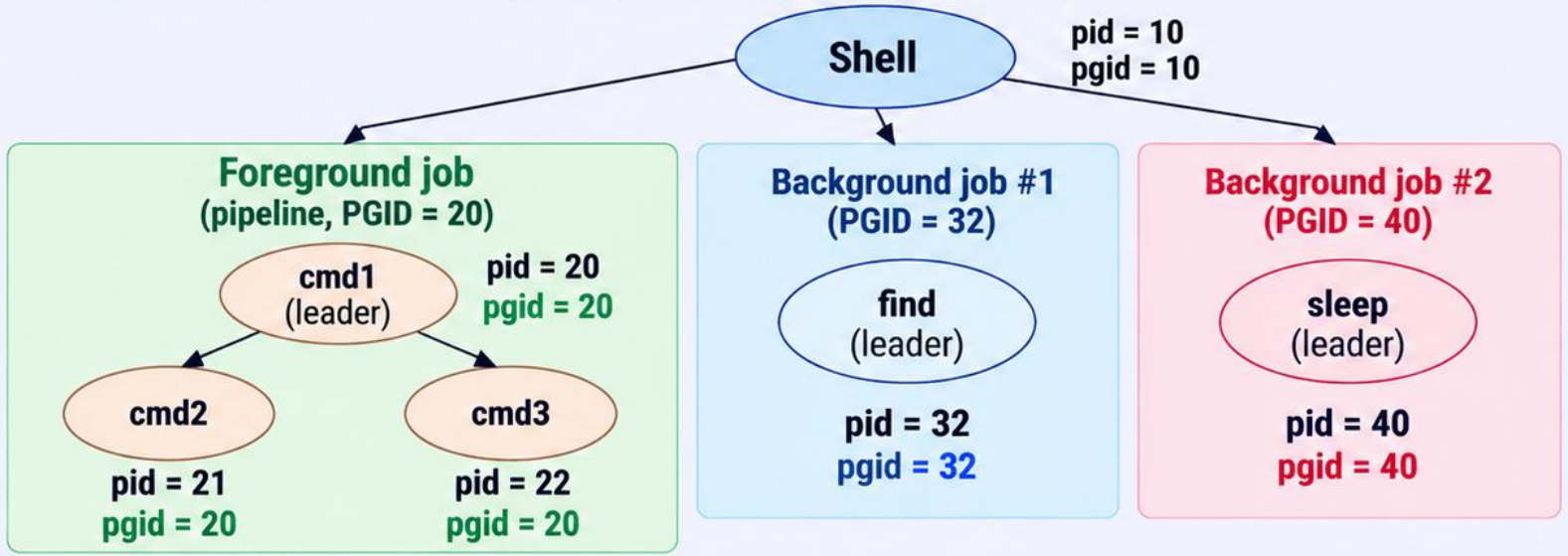
One job (same PGID)

```
$ cat access.log | grep ERROR | sort > result.txt ;
  find / -name "*.tmp" & sleep 100 &
```

Separate jobs (different PGIDs)

2 The shell creates a separate process group for each job

- Every process belongs to exactly one process group (PGID)
- The PGID equals the PID of the group leader
- The group leader is typically the first process in the job

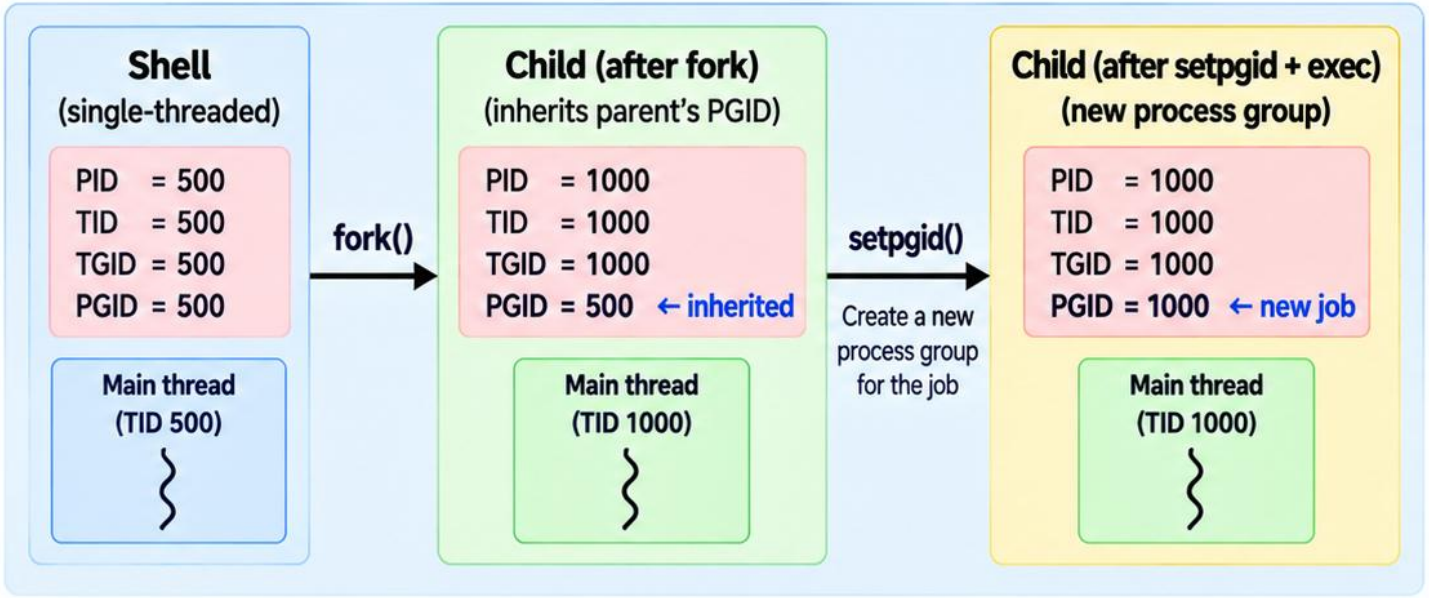


3 APIs

`getpgrp()`
Return process group of current process

`setpgid()`
Change process group of a process

PID (TGID) / TID and PGID in Linux

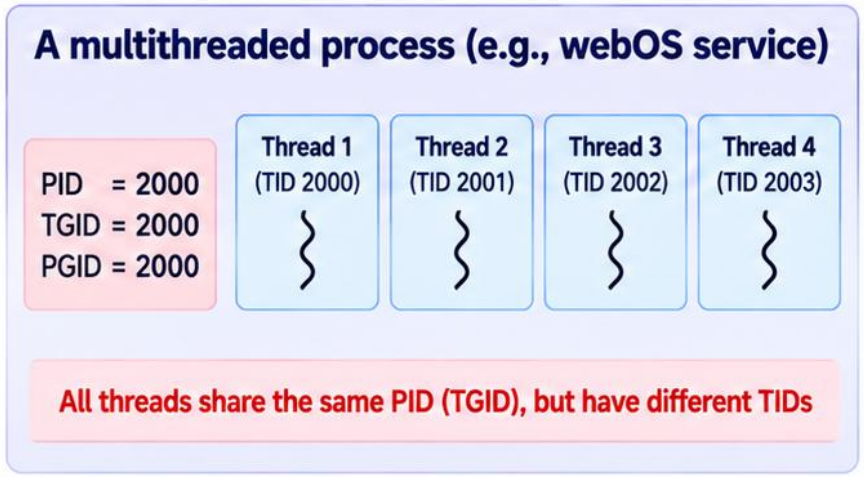


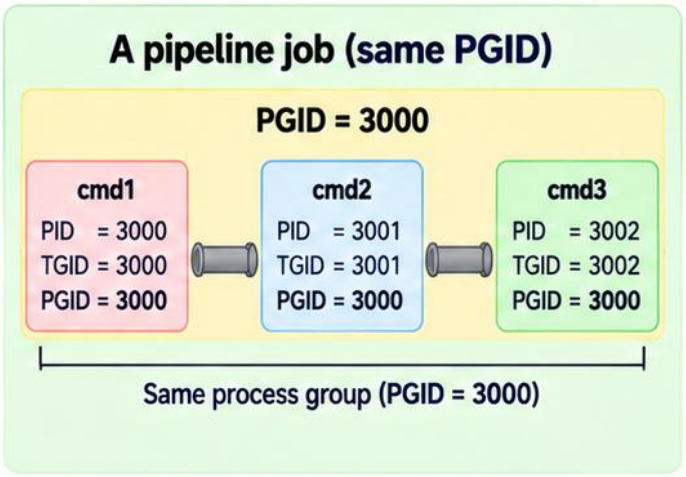
PID
- **Process ID** (unique per process)
- Identifies a process (seen in user space)

TID
- **Thread ID** (unique per thread)
- Each thread in a process has its own TID

TGID
- **Thread group ID** (same as the process's PID)
- All threads in the same process share TGID

PGID
- **Process group ID**
- Groups related processes as one job
- All processes in a pipeline (or job) share the same PGID





Key Points

- A process can have multiple threads (multiple TIDs).
- All threads in a process share the same **TGID** (which is the process's PID).
- A shell uses **setpgid()** to create a new process group for each job (e.g., a pipeline).
- All processes in the same pipeline belong to the same PGID.
- PGID is used for job control (e.g., terminal signals such as Ctrl+C).

Generating Signals with /bin/kill Program

- **/bin/kill** generates a signal to a process or a process group
  - **Positive PID** → send the signal to one process
  - **Negative PID** → send the signal to the process group whose PGID is |PID|

Examples

1. **/bin/kill -9 24818**
   Send SIGKILL to process 24818 only

2. **/bin/kill -9 -24817**
   Send SIGKILL to every process in process group 24817

1 Start a program (forks 16)

```
linux> ./forks 16
Child1: pid=24818 pgid=24817
Child2: pid=24819 pgid=24817
...
linux> ps
  PID  TTY          TIME  CMD
24788 pts/2    00:00:00  tcsh
24817 pts/2    00:00:00  forks (group leader)
24818 pts/2    00:00:02  forks
24819 pts/2    00:00:02  forks
24820 pts/2    00:00:00  ps
...
```

Same process group (PGID = 24817)

2 Kill the entire process group

```
linux> /bin/kill -9 -24817

linux> ps
  PID  TTY          TIME  CMD
24788 pts/2    00:00:00  tcsh
24823 pts/2    00:00:00  ps
```

All processes in PGID 24817 are gone

Generating Signals with kill Function

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void fork12(void)
{
    pid_t pid[N];
    int i;
    int child_status;

    for (i = 0; i < N; i++) {
      if ((pid[i] = fork()) == 0) {
          /* Child: Infinite loop */
          while (1) ;
      }
    }

    for (i = 0; i < N; i++) {
        printf("Killing process %d\n", pid[i]);
        kill(pid[i], SIGINT);
    }

    for (i = 0; i < N; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                   wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}
```

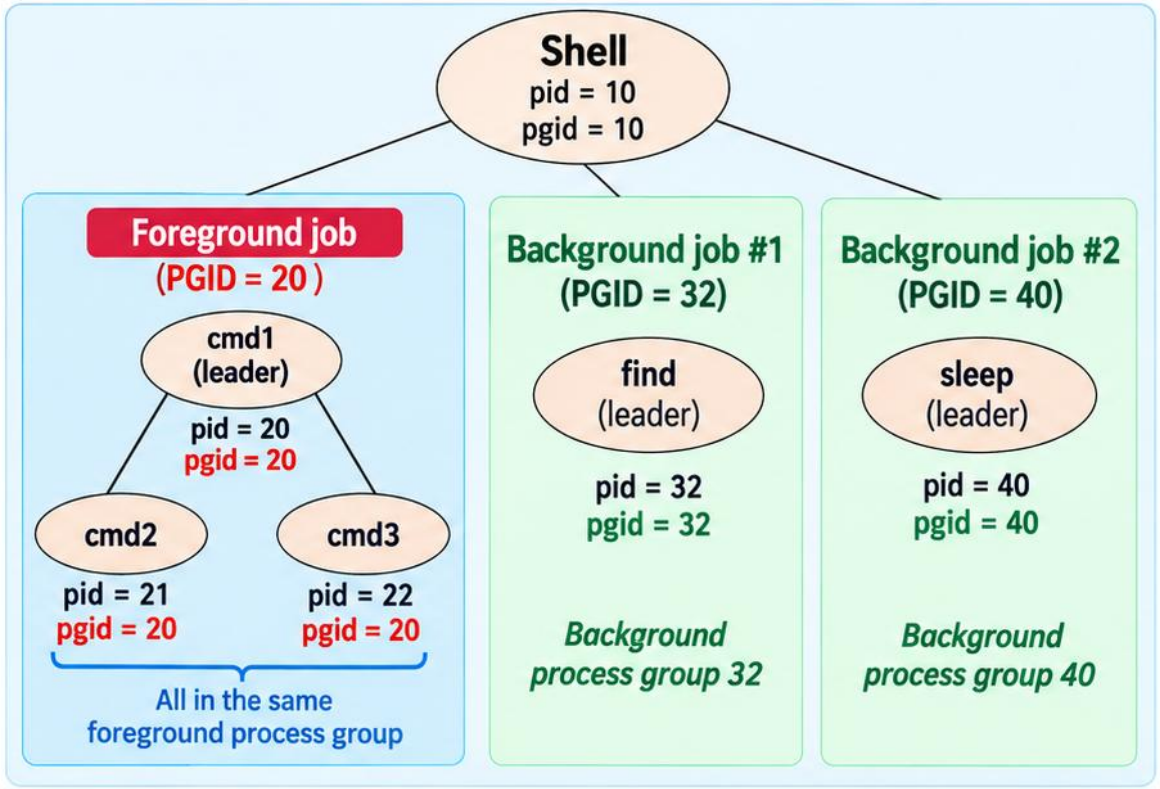
N = 5

```
→  14-ecf-procs ./forks 12
Killing process 24526
Killing process 24527
Killing process 24528
Killing process 24529
Killing process 24530
Child 24527 terminated abnormally
Child 24530 terminated abnormally
Child 24529 terminated abnormally
Child 24528 terminated abnormally
Child 24526 terminated abnormally
→  14-ecf-procs
```

Parent: send SIGINT to children

Parent: reap children

Generating Signals from the Keyboard



1 **Typing CTRL-C (CTRL-Z) causes the kernel to generate a SIGINT (SIGTSTP) to every process in the foreground process group**

CTRL-C →	SIGINT (2)	Default action: terminate each process
CTRL-Z →	SIGTSTP (20)	Default action: stop (suspend) each process

2 **Background jobs are not affected by CTRL-C / CTRL-Z**

- They keep running even if you press CTRL-C at the shell prompt
- If you want to stop them, you must send a signal explicitly

3 **Sending Signals Manually**

`kill(pid, SIGINT)`	`kill(-pgid, SIGINT)`	`kill %1`
Send a signal to one process only	Send a signal to the entire process group (negative sign)	Shell-level shortcut (kill job number 1)

Key Points

- CTRL-C / CTRL-Z → foreground group only
- Background jobs keep running
- Use kill, negative PGID, or %job to signal background jobs

Example of Ctrl-C and Ctrl-Z

1

```
bluefish> ./forks 17
Child: pid=28108 pgid=28107
Parent pid=28107 pgid=28107
^Z   (type Ctrl-Z)
Suspended
```

← **Ctrl-Z stops the foreground job**

2

```
bluefish> ps w
  PID    TTY      STAT     TIME    COMMAND
27699    pts/8    Ss       0:00    -tcsh
28107    pts/8    T        0:01    ./forks 17   ← Parent (stopped)
28108    pts/8    T        0:01    ./forks 17   ← Child (stopped)
28109    pts/8    R+       0:00    ps w
```

3

```
bluefish> fg
./forks 17
^C   (type Ctrl-C)
```

4

```
bluefish> ps w
  PID    TTY      STAT     TIME    COMMAND
27699    pts/8    Ss       0:00    -tcsh
28110    pts/8    R+       0:00    ps w
```

After Ctrl-C, the job is gone

STAT (process state) Legend

First letter (process state)

R	**running** (on CPU or runnable)
S	**sleeping** (interruptible sleep)
T	**stopped** (e.g., by Ctrl-Z, SIGTSTP)

Second letter (additional flags)

s	**session leader** (has a controlling terminal)
+	**foreground process group** (in the foreground)

See **"man ps"** for more details

Installing Signal Handlers

signal()

```
#include <signal.h>

/* handler: function or SIG_IGN or SIG_DFL */
sighandler_t signal(int signum, sighandler_t handler);
```

- Simple interface, historically used
- **Problems:**
  - Behavior differs across Unix systems
  - May reset to default after handler returns
  - Cannot control signal mask or flags
  - Not suitable for robust signal handling

sigaction() (recommended)

```
#include <signal.h>
int sigaction(int signum,
              const struct sigaction *act,
              struct sigaction *oldact);
```

```
struct sigaction {
   void      (*sa_handler)(int);  /* or sa_sigaction */
   sigset_t  sa_mask;             /* block during handler */
   int       sa_flags;            /* e.g., SA_RESTART */
   void      (*sa_sigaction)(int, siginfo_t *, void *);
};
```

- **Well-defined and portable**
- **Specify signal mask** (sa_mask)
- **Control behavior via flags**
  - SA_RESTART: restart interrupted syscalls
  - SA_SIGINFO: get additional info
  - …
- **Does not reset to default**
- **Recommended** for modern code

Default action (if no handler is installed)

Signal	Default action	Description
SIGINT	Terminate	Ctrl-C from terminal
SIGTERM	Terminate	Normal termination request
SIGKILL	Terminate	Cannot be caught or ignored
SIGSTOP	Stop	Cannot be caught or ignored
SIGSEGV	Terminate	Invalid memory access
SIGCHLD	Ignore	Child terminated (usually ignored)
…	…	…

Example: install a handler for SIGINT

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int signo) {
   write(STDOUT_FILENO, "Caught SIGINT\n", 14);  /* async-signal-safe */
}

int main() {
   struct sigaction sa;
   sa.sa_handler = handler;
   sigemptyset(&sa.sa_mask);
   sa.sa_flags = 0;            /* or SA_RESTART */
   if (sigaction(SIGINT, &sa, NULL) == -1) {
      perror("sigaction");
      return 1;
   }
   while (1) pause();          /* wait for signals */
}
```

Signal Handling Example

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sigint_handler(int sig) /* SIGINT handler */
{
    const char msg[] = "Caught SIGINT\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    _exit(0);  /* use _exit(): async-signal-safe */
}

int main(void)
{
    struct sigaction sa;

    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;   /* Restart syscalls if possible */

    /* Install the SIGINT handler */
    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction error");
        exit(1);
    }

    /* Wait for the receipt of a signal */
    while (1) {
        pause();  /* sleep until a signal is caught */
    }
    return 0;
}
```



Signal handlers require special care

- They can run **asynchronously** during normal execution
- Only ***async-signal-safe*** functions should be called
- Examples (O = safe, X = NOT safe):

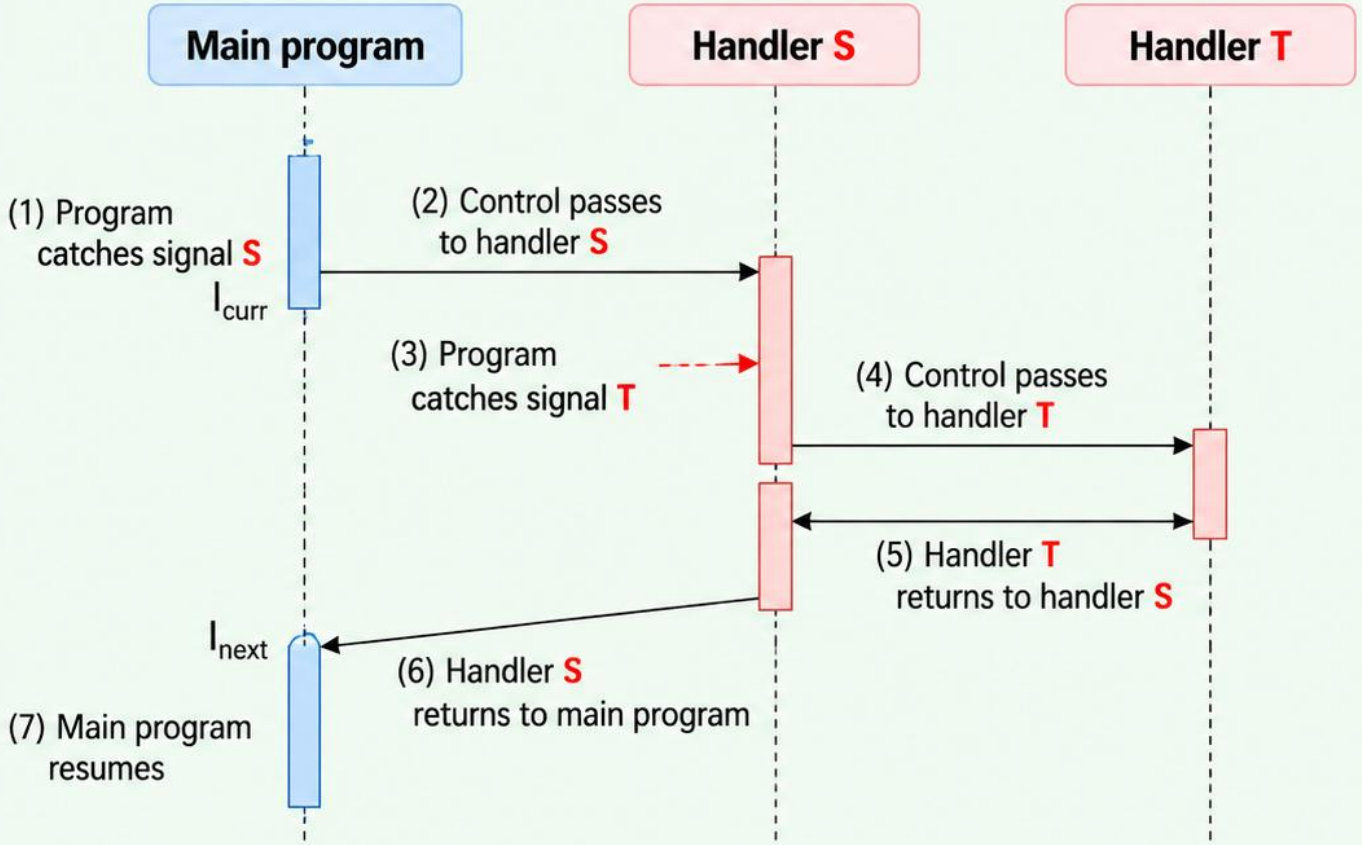
```
write()   : O
printf()  : X
_exit()   : O
exit()    : X
```

- Refer to the POSIX list of async-signal-safe functions.

Nested Signal Handlers

- **Handlers can be interrupted by other handlers**
  - The signal currently being handled is normally blocked while its handler executes
  - Other unblocked signals may still be delivered

While Handler S is running, the kernel may regain control (e.g., via a system call, interrupt, or context switch) and deliver another pending, unblocked signal T before returning to user mode.

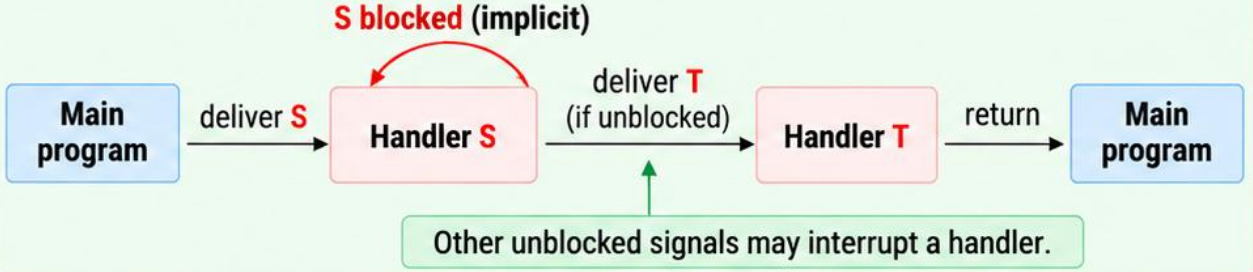


Blocking and Unblocking Signals

Implicit blocking mechanism

- By default, the signal being handled is blocked while its handler executes
- E.g., a **SIGINT** handler can't be interrupted by another **SIGINT**

While a handler for signal S is running, S is blocked automatically, but other unblocked signals (e.g., T) may still be delivered.



Explicit blocking and *unblocking* mechanism

- Use **sigprocmask()** to manipulate the signal mask for the calling process (or thread).

```
sigset_t mask, prev_mask;

sigemptyset(&mask);
sigaddset(&mask, SIGINT);

/* Block SIGINT and save previous blocked set in prev_mask */
sigprocmask(SIG_BLOCK, &mask, &prev_mask);

/* Code region that will not be interrupted by SIGINT */
...

/* Restore previous blocked set (unblock SIGINT) */
sigprocmask(SIG_SETMASK, &prev_mask, NULL);
```

sigprocmask()

```
int sigprocmask(int how,
                const sigset_t *set,
                sigset_t *oset);
```

Operations (how)

how	Description
SIG_BLOCK	Add the signals in *set* to the current signal mask
SIG_UNBLOCK	Remove the signals in *set* from the current signal mask
SIG_SETMASK	Replace the current signal mask with *set*

Supporting functions

- **sigemptyset**(sigset_t *set)
  – Create an empty set
- **sigfillset**(sigset_t *set)
  – Add every signal number to the set
- **sigaddset**(sigset_t *set, int signo)
  – Add a signal signo to the set
- **sigdelset**(sigset_t *set, int signo)
  – Delete a signal signo from the set

Why not using SIG_UNBLOCK?

Using **SIG_UNBLOCK** might inadvertently unblock signals that were already blocked before this critical section.

Guidelines for Writing Safe Handlers

G0: Keep your handlers as simple as possible

- Do the minimum work in a handler
- e.g., set a global flag and return
- Defer complex processing to the main program

```
void handler(int sig)
{
        flag = 1;   /* keep it simple */
}
```

G1: Call only async-signal-safe functions in your handlers

- A function is async-signal-safe if it is reentrant or atomic with respect to signals, and it does not call any non-async-signal-safe functions
- Use only functions defined as async-signal-safe by POSIX

Examples (safe)	Examples (NOT safe)
• write() • _exit() • wait(), waitpid() • sleep() • kill() •	• printf() • sprintf() • malloc() • exit() • ....

G2: Save and restore errno on entry and exit

- Handlers may be interrupted by other handlers
- Save errno at the beginning and restore it before returning
- Prevents other handlers (or the handler itself) from overwriting your value

```
void handler(int sig)
{
    int olderrno = errno;
    ....
    errno = olderrno;
}
```

G3: Protect shared data with signal masking

- Temporarily block all signals when accessing shared data structures
- Prevents possible corruption

```
sigset_t mask, prev_mask;

sigemptyset(&mask);
sigaddset(&mask, SIGINT);
sigprocmask(SIG_BLOCK, &mask, &prev_mask);  /* critical section */
....
sigprocmask(SIG_SETMASK, &prev_mask, NULL);  /* restore */
```

G4: Declare global variables as volatile

- Prevent the compiler from keeping the variable in a register (optimization)
- Use volatile for variables modified in a handler

```
volatile sig_atomic_t flag = 0;
void handler(int sig)
{
    flag = 1;
}
```

G5: Use sig_atomic_t for shared variables

- Use sig_atomic_t for variables that are only read or written atomically (e.g., flag = 1, not flag++)
- Such flags do not need additional protection like other global variables

```
volatile sig_atomic_t flag = 0;
int main(void)
{
    while (!flag) ;  /* wait */
}
```

Correct Signal Handling

Standard signals are not queued

- For each signal type, one bit indicates whether or not a signal is pending.
- If multiple identical signals are generated, they may be *coalesced* into a single pending signal.

You can't use signals to count events, such as children terminating.

Multiple child exits may generate only one SIGCHLD delivery. Therefore, the handler must reap all terminated children.

Incorrect handler (reaps only one child)

```
void child_handler(int sig)
{
   int olderrno = errno;
   pid_t pid;

   pid = wait(NULL);       /* reap only one child */
   if (pid > 0)
      count--;             /* decrement once */

   errno = olderrno;
}
```

If multiple children terminate before the handler runs, the handler may execute only once, reaping only one child.

Correct handler (reap all terminated children)

```
void child_handler(int sig)
{
   int olderrno = errno;
   pid_t pid;

   /* Reap all available terminated children */
   while ((pid = waitpid(-1, NULL, WNOHANG)) > 0) {
      count--;             /* decrement for each child */
   }

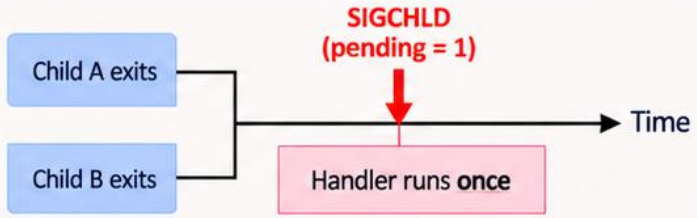
   errno = olderrno;
}
```

Loop with waitpid(-1, ..., WNOHANG) to reap all terminated children even if only one SIGCHLD is delivered.

Main program (parent)

```
volatile sig_atomic_t count = 0;
int main(void)
{
    int i, pid;
    count = N;              /* number of children */
    signal(SIGCHLD, child_handler);
    for (i = 0; i < N; i++) {
        pid = fork();
        if (pid == 0) {     /* child */
            sleep(1);       /* do some work */
            exit(0);
        }
    }
    while (count > 0)       /* wait for all children */
        ;                   /* (e.g., spin or pause) */
    return 0;
}
```

Two children exit nearly simultaneously



Two child exits generate only one SIGCHLD delivery (because standard signals are not queued). The handler must reap both children.

Result (N = 5)

```
ecf-signals> ./forks 5
Handler reaped child
Handler reaped child
Handler reaped child
Handler reaped child
Handler reaped child
ecf-signals>
```

Even if multiple children exit at the same time, the correct handler reaps all of them.

Exactly N lines
(5 lines for N = 5).

Key takeaway

- SIGCHLD notifications can be lost/coalesced.
- **Signals** tell you "**something may** have changed," not how many children terminated.
- Always use a loop with waitpid(-1, ..., WNOHANG) in the handler.

Explicitly Waiting for Signals

1 Main program (parent)

```c
volatile sig_atomic_t child_done = 0;

void sigchld_handler(int sig)
{
    int olderrno = errno;
    pid_t pid;
    while ((pid = waitpid(-1, NULL, WNOHANG)) > 0)
        child_done = 1;
    errno = olderrno;
}

int main(int argc, char **argv)
{
    struct sigaction sa;
    sigset_t mask, prev;

    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGCHLD, &sa, NULL);

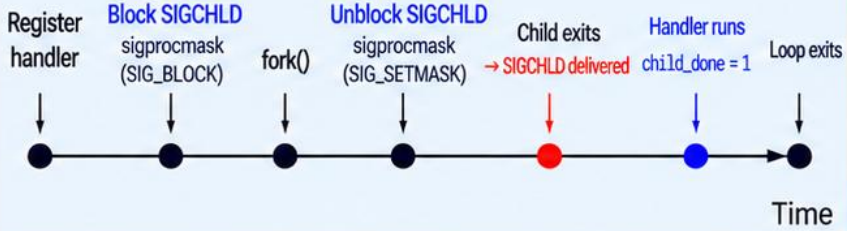
    sigemptyset(&mask);
    sigaddset(&mask, SIGCHLD);
    sigprocmask(SIG_BLOCK, &mask, &prev);

    pid_t pid = fork();
    if (pid == 0) {
        sleep(1);
        _exit(0);
    }

    sigprocmask(SIG_SETMASK, &prev, NULL);

    while (!child_done) ;  /* explicitly wait */
    printf("Child finished\n");
    return 0;
}
```

2 Execution timeline

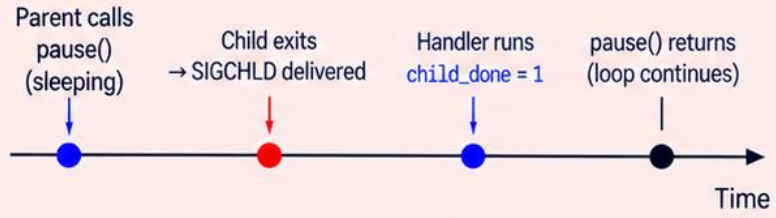


3 Using pause() instead of busy waiting

Replace the busy loop with pause(), which sleeps until a signal is delivered.

```c
while (!child_done)
    pause();
```

1. Parent calls pause() and goes to sleep.
2. When the child exits, the kernel delivers SIGCHLD and the handler runs (reaping the child and setting child_done = 1).
3. pause() returns and the loop checks child_done, then exits.



Problems with explicit wait

1 Busy waiting (without pause())

- while (!child_done) ; continuously spins and wastes CPU cycles.
- Inefficient, especially if the child runs for a long time.

2 Race condition (lost wakeup)

- SIGCHLD may arrive after checking child_done but before calling pause().
- pause() then sleeps even though the signal has already been delivered.
- The process may sleep forever.

3 Unnecessary wakeups

- pause() returns for any signal, not only SIGCHLD (e.g., SIGINT).
- The loop may repeat, causing extra wakeups and CPU overhead.

4 Slow response (with sleep())

- Using sleep() avoids busy waiting but can be very slow (latency up to 1 second or more).

Key takeaway

- Block **SIGCHLD** before **fork()**, unblock after fork(), then explicitly wait (busy loop or pause()).
- Using **pause()** is simple, but has serious problems: race condition, unnecessary wakeups, and potentially sleeping forever.

Waiting for Signals with sigsuspend

1 Main program (parent)

```c
volatile sig_atomic_t child_done = 0;

void sigchld_handler(int sig)
{
    int olderrno = errno;
    pid_t pid;
    while ((pid = waitpid(-1, NULL, WNOHANG)) > 0)
        child_done = 1;
    errno = olderrno;
}

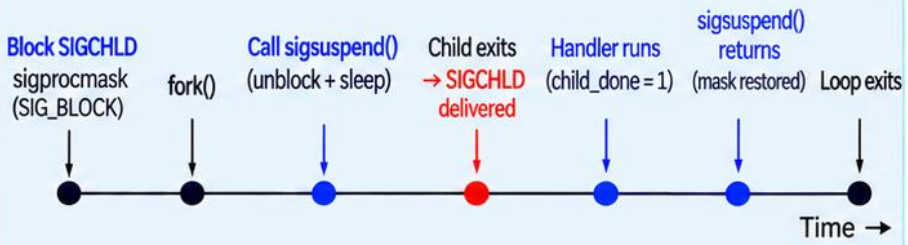
int main(int argc, char **argv)
{
    struct sigaction sa;
    sigset_t mask, prev;

    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGCHLD, &sa, NULL);

    /* Block SIGCHLD before forking */
    sigemptyset(&mask);
    sigaddset(&mask, SIGCHLD);
    sigprocmask(SIG_BLOCK, &mask, &prev);

    pid_t pid = fork();
    if (pid == 0) {
        execlp("/bin/ls", "ls", NULL);
        _exit(127);
    }
    /* Wait for SIGCHLD using sigsuspend */
    while (!child_done)
        sigsuspend(&prev);
    /* Optionally restore original mask */
    sigprocmask(SIG_SETMASK, &prev, NULL);
    printf("Child finished\n");
}
```

2 Execution timeline



SIGCHLD blocked (before sigsuspend)	SIGCHLD unblocked during sigsuspend()	Mask automatically restored after return

3 How this works

1. Install the SIGCHLD handler and block SIGCHLD before fork(), so the signal cannot be delivered early.
2. Call **sigsuspend**(&prev) inside the loop. This atomically replaces the mask with prev (unblocks SIGCHLD) and puts the process to sleep.
3. When the child exits, the kernel delivers SIGCHLD, the handler runs and sets child_done = 1.
4. **sigsuspend()** returns after the handler returns, restoring the previous mask (SIGCHLD blocked).
5. The loop condition is rechecked. When child_done = 1, the loop exits.
6. Optionally, restore the original mask using **sigprocmask**(SIG_SETMASK, &prev, NULL).

! Assume SIGCHLD was initially unblocked (not blocked before blocking it here).

4 Advantages of sigsuspend()

1 Eliminates the race condition

- No window between unblocking SIGCHLD and sleeping.
- Mask replacement and sleep are done atomically by sigsuspend().

2 No lost wakeup

- If SIGCHLD is delivered just before sigsuspend() is called, it is delivered immediately and sigsuspend() returns without sleeping.

3 No unnecessary CPU usage

- The process sleeps until a signal is delivered (unlike busy waiting).

Key takeaway

- Use **sigsuspend()** with a mask that unblocks SIGCHLD to safely wait for the child process.
- After the loop, optionally restore the original signal mask.

Signals You Actually See on webOS / Linux

Signal	Default Action	What It Means (Typical Cause)	Example (webOS/Linux)
SIGTERM 15	Terminate	**Termination request** (e.g., service shutdown; can be caught for graceful shutdown)	kill <pid> (default) kill -TERM <pid> systemctl stop <service> (application can handle for graceful shutdown)
SIGKILL 9	Terminate (immediate)	**Forceful kill (cannot be caught)** (e.g., unresponsive process)	kill -9 <pid> (force kill)
SIGSEGV 11	Terminate	**Invalid memory access** (e.g., NULL dereference, out-of-bounds)	native service crash (check core dump)
SIGABRT 6	Terminate	**Abort due to assertion failure** (e.g., abort(), assert())	assertion failure (native service)
SIGCHLD 17	Ignore (by default)	**Child process state changed** (exited, stopped, continued)	child exit (handle with wait())
SIGINT 2	Terminate	**Sent from terminal** (e.g., Ctrl+C)	Ctrl+C (shell, foreground process)
SIGSTOP 19	Stop (cannot be caught)	**Stop the process** (e.g., debugging)	kill -STOP <pid> (debugging)
SIGCONT 18	Continue	**Continue a stopped process** (resumed)	kill -CONT <pid> (resume)

Where Signals Come From

User / Shell
e.g., Ctrl+C → SIGINT
kill <pid> → SIGTERM

Process / Kernel
e.g., child exit → SIGCHLD

System / Manager
e.g., systemd stop → SIGTERM

Kernel / Runtime
e.g., protection fault → SIGSEGV

Key Takeaways

1. Signals are used by the kernel to notify processes about events.
2. They are lightweight and asynchronous.
3. Some signals can be caught and handled, others cannot (e.g., SIGKILL, SIGSTOP).

SIGTERM: Graceful Shutdown of a Native Service in webOS

SIGTERM (15): Termination Request

- Default action: Terminate
- Can be caught by the process to perform graceful shutdown.
- Used to request a clean shutdown, not a forced kill.

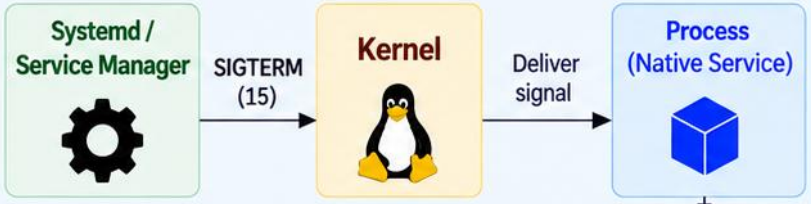
Example: Handling SIGTERM (sigaction)

C (glib / webOS)

```c
#include <signal.h>
#include <stdatomic.h>
#include <unistd.h>

static volatile sig_atomic_t g_quit = 0;
static void term_handler(int sig) {
    g_quit = 1;    // set flag only (async-signal-safe)
}
int main() {
    struct sigaction sa = {0};
    sa.sa_handler = term_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    if (sigaction(SIGTERM, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }
    while (!g_quit) {
        /* main loop: do work */
    }
    cleanup_resources();
    return 0;
}
```

Graceful Shutdown Flow



1. **Signal Handler** — Set termination flag only (return immediately)
2. **Main Loop Notices** — Check flag and stop work (terminate cleanly)
3. **Cleanup Resources** — Close files / sockets / DB, release IPC, etc.
4. **Exit Normally** — return from main()

Why this pattern?

- Handler runs in async context (keep it simple).
- Only set a flag in handler (async-signal-safe).
- Do real cleanup work in the normal execution path.

How to Trigger (webOS/Linux)

From device shell

```
kill -TERM <pid>
```

(request termination)

From systemd

```
systemctl stop <service>
```

(send SIGTERM)

Best Practices

- Use sigaction() (or signalfd) for reliable handling.
- Keep the signal handler short (set a flag only).
- Do cleanup in normal context (not in handler).
- Release resources (IPC, files, threads, DB, etc.).
- Test with: kill -TERM <pid> or systemctl stop.

Typical Use in webOS

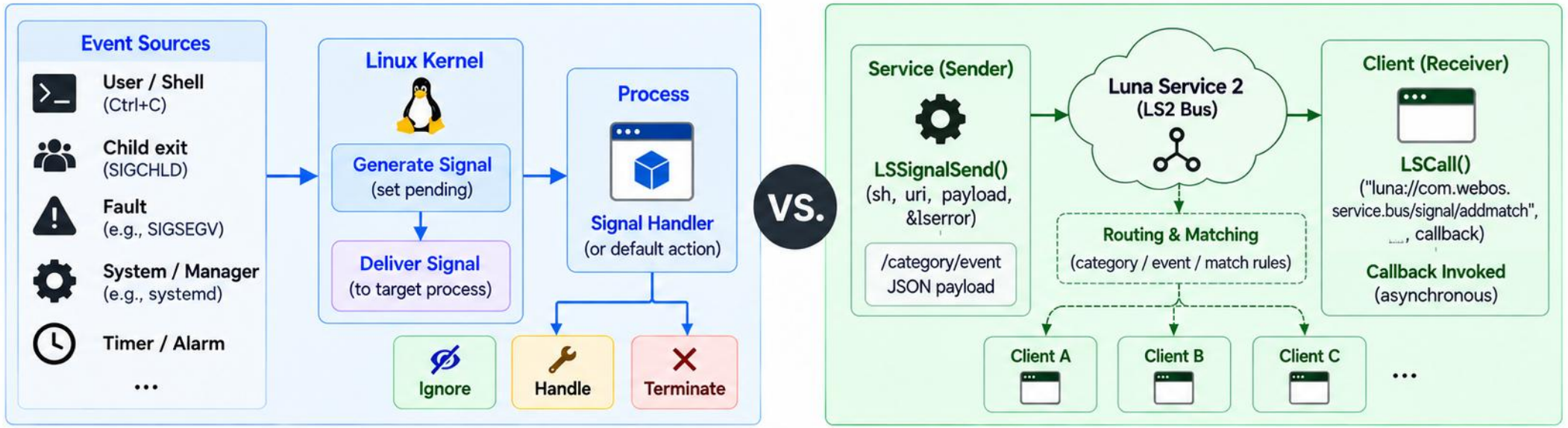
- Native services (webos_daemon / webos_systemd) receive SIGTERM on stop/restart.
- Perform cleanup and exit normally to keep the system stable and responsive.

If We Ignore SIGTERM

Systemd sends SIGTERM → Process ignores it → After timeout, systemd sends SIGKILL (9) → Process is killed immediately (no cleanup) → Possible data loss (inconsistent state)

Good services don't just stop — they **finish what matters** and leave the system in a **healthy state.**

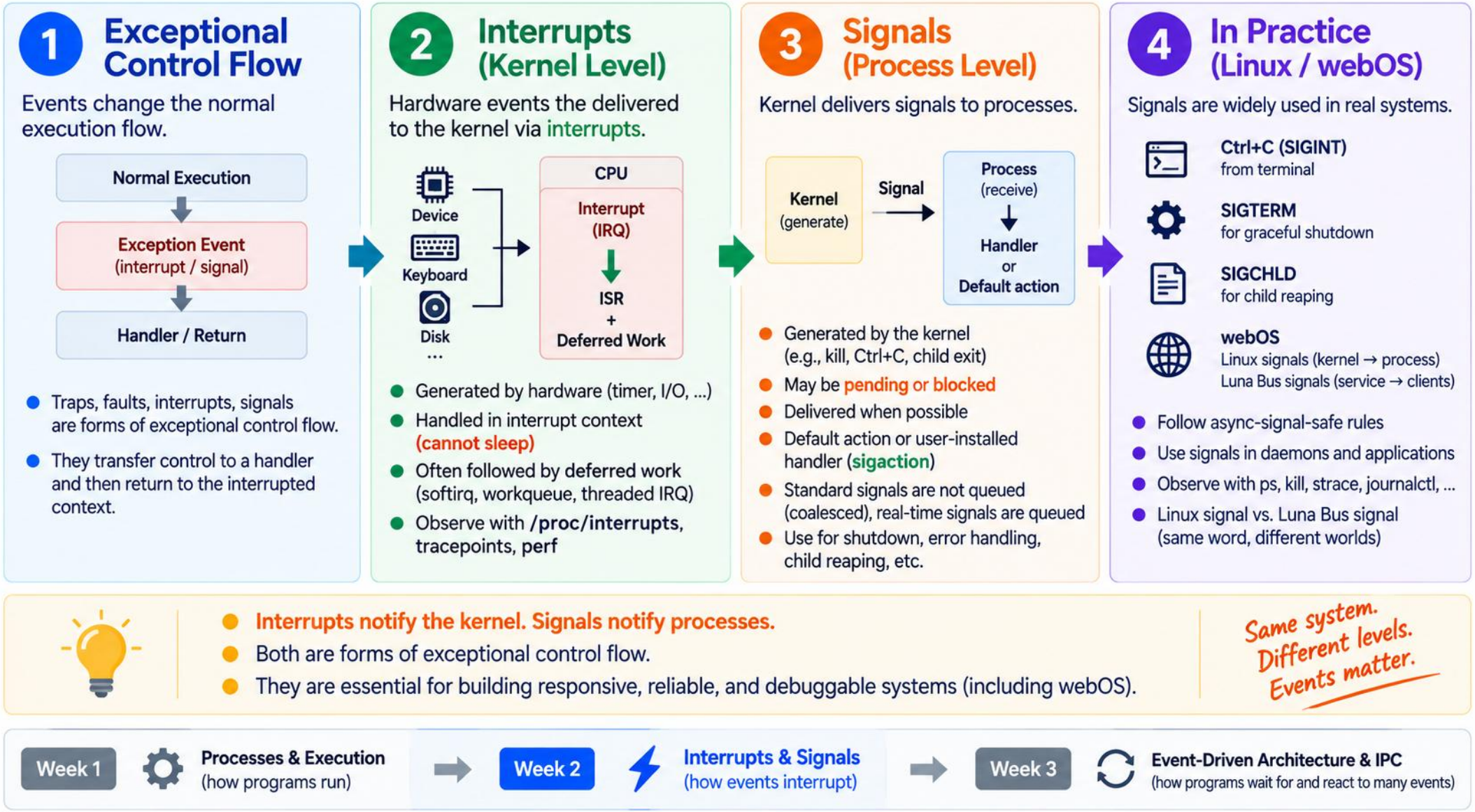
Linux Signal vs. Luna Bus Signal



Aspect	Linux Signal	Luna Bus Signal (LS2)
Purpose	Process control, error notification, exceptional events	Normal IPC event notification between services/clients
Path	Kernel → Process	Service → LS2 Bus → Clients
Transport	In-kernel mechanism (per-process)	User-space message bus (domain socket + LS2 protocol)
Target	A specific process (or process group)	All clients that registered matching rules
Delivery	Not queued (per signal number), often coalesced	Queued / delivered to all matched clients
Typical Use	SIGTERM, SIGSEGV, SIGCHLD, ...	UI updates, state changes, event-driven communication
Example	`kill -TERM <pid>`, Ctrl+C (SIGINT)	`LSSignalSend(sh, "luna://com.webos.app/update/volume", payload, &lserror)`

Same word "signal", different worlds.
Linux signals are for **process control** and **exceptional conditions.** Luna Bus signals are for normal **inter-service communication** on webOS.

Week 2 Summary



End of Week 2

Sungyong Park, Ph.D. | parksy@sogang.ac.kr



